

Introducing TEA OS - A Minimal Operating System Implemented Using the TEA Programming Language

Willrich J. Lutalo*

joewillrich@gmail.com, jwl@nuchwezi.com

May 28, 2026

Abstract

This paper briefly introduces the TEA Operating System (TEA OS); a compact software operating environment runtime built on top of and leveraging the Transforming Executable Alphabet programming language. TEA OS packages a minimal shell, a set of core utilities for doing math, TEA programming and web I/O, as well as a robust, directory-less persistence layer in form of a file system that has a suite of file I/O commands familiar to users of Linux, Unix and MS-DOS. TEA OS intentionally targets userland portability rather than traditional kernel responsibilities, a distinction we clarify and formalize in this paper. We present a preserved verbatim source listing of the current (v1.1.0) system implementation in the TEA language. To illustrate usage and pedagogy, the paper includes an introductory note, the complete source code listing, and a sample interactive session dump showing a user driving the TEA shell. Given that the TEA OS is currently shipped alongside the TEA language as part of the standard TEA package and programs, it is hoped that those evaluating the usability, relevance and power of TEA programming can leverage this example general case to truly come to appreciate the innovation in the language known as the Transforming Executable Alphabet (TEA).

Keywords: Transforming Executable Alphabet, TEA, Applying TEA, Minimal Operating System, Text-Processing; Software Operating Environments, Preprint, Source Code.

1 Introduction

Considering all the potential ways a [general-purpose] programming language might be utilized, realizing an operating system is among the most critically

*Inventor of the TEA language, also currently a volunteering & Independent Principal Investigator at Nuchwezi Research — <https://nuchwezi.com>

important ends one might consider [1]. Having already finished introducing the world to the Transforming Executable Alphabet (TEA) language in [2], it is indeed necessary to go ahead and likewise formally introduce some of the special cases it has been well applied to.

The TEA Operating System (TEA OS), also sometimes referred to as “TOS”, is a software operating environment (SOE) [3] meant to allow for more than just writing, storing and running [TEA] programs; it allows for the writing and evaluation of mathematical expressions; fetching, saving, processing, and presentation of web-based/network-accessible resources; as well as ready access-to a user-aware, extensible and minimal, self-documenting shell known as the TEA OS Shell (perhaps better we start calling it “TOSH” to keep with traditional shell-naming conventions).

It shall be worth noting that TEA OS was designed with a different set of assumptions than conventional operating systems: rather than assuming privileged hardware access and low-level languages, TEA OS assumes a TEA runtime baseline and prioritizes minimalism and portability. This design enables TEA OS to be expressed compactly and to run in environments such as web browsers without a virtual machine; it also means that TEA OS should be read as a userland runtime and service stack unless explicitly extended with kernel-level primitives.

The complete design, justification, specification and demonstrations of the current TEA OS implementation are already well covered in the relevant chapter in the TEA language manual [1]. We shall not waste time and space reproducing those bits here then. However, it should be worth noting that, from the onset, the design of TEA OS was without doubt inspired by its inventor’s awareness of the ideas underlying popular traditional mainstream operating systems such as Unix, Linux and the currently almost defunct MS-DOS.

Next, we are going to consider the initial, complete specification of the TEA OS in **Section 2** as first laid down in [1]. We shall consider the TEA OS architecture in **Section 3**, and shall discuss how TEA OS relates to, differs from traditional operating system kernels in **Section 4**. **Section 5** presents the verbatim form of the complete source code of **TEA OS v1.1.0** also which can be found in its original, but also up-to-date form via https://bit.ly/run_teaos, and then we shall reproduce a note originally introducing v1.1.0 of the OS in **Section 6**, as well as a verbatim session dump — exported from the associated TOSH in **Section 7**. We then conclude in **Section 8** with key observations made as well as what the potential future directions are.

2 The TOS Specification

Without the sanction of some working-group or committee, we have thought-up and drafted the following minimal specification as the essential basis of the **TEA Operating System**:

TOS Specification:

The TEA Operating System is specified based on 4 core capabilities:

1. File Input/Output Service
2. [TEA] Programming Service
3. Basic Utility Service
4. Help/Documentation Service
5. UI/Interface Service

Moreover, and perhaps very worthwhile to note, we learn from one of the reference books on Linux — a globally popular, and dominant free operating system — that besides the core (or rather “kernel”) of an operating system, it is quite important to be clear on what services and utilities render the operating system usable. See what Ellen Siever and Stephen Spainhour et al. have to say about this:

The Linux kernel itself was originally designed by Linus Torvalds at the University of Helsinki in Finland and later developed through collaboration with countless volunteers worldwide. By "kernel," we mean the core of the operating system itself---not the applications (such as the compiler, shells, and so forth) that run on it. Today, the term "Linux" is often used to mean the kernel as well as the applications and complete system environment.

—Ellen Siever and Stephen Spainhour et al., *Linux in a Nutshell: A Desktop Quick Reference*, 2000[4]

And with that clarification out of the way, we then embark on specifying the details of our operating system — and somewhat, to again reflect the manner and approach of [4] in clarifying what makes Linux tick as well as how it is used, we shall especially approach the specification as though we were laying down a specification of the interaction interface that users of TOS rely upon to use the operating system for arbitrary tasks. The four categories or capabilities laid down in the mini-specification above shall guide us...

1. **File System:** create, name, rename, copy, lookup, preview, and delete file objects.

—[Basic FILE I/O Commands Interface]:

- **create** FILENAME $\leftarrow$ creates blank file with name FILENAME. Valid FILENAME should consist of ONLY letters a-zA-Z, numbers 0-9, underscore _ and/or a dot ..
 - **init** FILENAME $\leftarrow$ create a file using the given name and the currently available input/last-output (or the **print buffer**) or blank if none.
 - **write** FILENAME $\leftarrow$ write the contents of the current available input/last-output (or the **print buffer**) or blank if none, to a file with the specified name. If the file already exists, essentially overwrites its contents.
 - **exists** FILENAME $\leftarrow$ Show whether or not a file named FILENAME exists.
 - (**rename** | **rn** | **mv** | **move**) FILENAME NEWNAME $\leftarrow$ Change the name of an existing file from FILENAME to NEWNAME. WILL ALSO update the registered [user] programs list if FILENAME was a registered program.
 - (**copy**|**clone**) FILE1 FILE2 $\leftarrow$ Copies contents of FILENAME to FILE2 creating the destination if it doesn't exist.
 - **find** PATTERN $\leftarrow$ Display list of ONLY files matching regular expression PATTERN.
 - (**show** | **cat** | **read**) FILENAME $\leftarrow$ Preview the contents of given file.
 - **export** FILENAME $\leftarrow$ Exit the system, outputting the contents of given file.
 - (**del** | **delete** | **rm**) PATTERN $\leftarrow$ Remove from storage, any and all files whose name matches PATTERN. CAN NOT DELETE registered user program files though.
 - **ls** | **list** | **dir** | **files** $\leftarrow$ Display list of ALL available files.
 - (**ls** | **list** | **dir** | **files**) PATTERN $\leftarrow$ Display list of ONLY files matching regular expression PATTERN.
2. **Programming Interface:** a command-line interface to find, run, or modify user programs or find and run system programs and commands.
- [**Basic Programming Commands Interface**]:
- (**runnable** | **register** | **add-program**) FILENAME $\leftarrow$ Recognize FILENAME object as an executable [user] program or command. FILENAME must already be an existing TEA file.
 - **programs** [PATTERN] $\leftarrow$ Show list of all available registered [TEA] user programs by their registered names. With PATTERN specified, ONLY show those programs whose names match regular expression PATTERN.

- `(isrunnable | canrun) FILENAME` $\leftarrow$ Show whether or not a file named FILENAME can be executed as a [TEA] program.
 - `(run | exec | execute) FILENAME` $\leftarrow$ Execute the given file record as a [TEA] program.
 - `(del-prog | delete-program | unregister) FILENAME` $\leftarrow$ Revokes status of FILENAME as an executable user program (doesn't actually delete the underlying file object though).
 - `(update | update-program) FILE1 FILE2` $\leftarrow$ Replace/override original program named FILE1 with contents of FILE2 (updates a program's code [in FILE1] preserving its existing name). Both files FILE1 and FILE2 must actually exist for this to work, but only FILE1 needs to be a registered program.
 - `PROGRAM` $\leftarrow$ Executes PROGRAM as a system command if it exists — first, if it is any of the standard commands in TEA OS, otherwise if it is a valid filename “PROGRAM” that is already recognized/registered as a user-created program — this later case similar to calling “run PROGRAM”, otherwise shows error message command-not-found.
 - `(help | man | whatis | info | about) PROGRAM` $\leftarrow$ displays help info about a given system command or user program.
3. **Basic Utilities:** access to basic useful commands such as being able to interact with the world outside TOS as well as do non-programming work from within TOS.
- [**Basic UTILITY Commands Interface**]:
- `(clear | reset)` $\leftarrow$ Resets and clears the current TEA OS session log since login.
 - `date` $\leftarrow$ return the current date.
 - `echo MESSAGE` $\leftarrow$ Simply print MESSAGE as is.
 - `exit` $\leftarrow$ ends the TOS session returning nothing.
 - `export` $\leftarrow$ Take the output of the last command or rather, current contents of the output buffer and export that outside the operating system (also terminates the operating system session).
 - `(fetch | web) URL` $\leftarrow$ fetch and output contents of the specified [online/network] resource.
 - `fetch URL FILENAME` $\leftarrow$ fetch and save contents at URL into file record named FILENAME.
 - `logout` $\leftarrow$ Resets username and system logs and ends the TOS session returning nothing.
 - `math EXPRESSION` $\leftarrow$ execute basic mathematics on the command-line as specified in EXPRESSION.

- **print** $\leftarrow$ Show or preview current output buffer contents (usually, output of last command). After clearing log, but also after exit/system shutdown/login also shows final output before the log was reset.
 - **tea CODE..** $\leftarrow$ take everything following the keyword “tea” as a TEA program, run and output the result: It’s a simple way to write and run basic TEA programs (TEA-one-liners) in TOS.
 - **time** $\leftarrow$ return the current time.
 - **timestamp** $\leftarrow$ return the current timestamp.
 - **username NAME** $\leftarrow$ Overrides current username with specified value.
 - **(whoami|name)** $\leftarrow$ Display the NAME of currently active USER.
4. **Help System:** a means to locate and read basic documentation about programs, system commands and the operating system.
—[**Basic HELP Commands Interface**]:
- **about** $\leftarrow$ Displays basic info about the currently running version of the TEA Operating System.
 - **help | man | whatis | info** $\leftarrow$ present menu for user to access help about all available [essential] commands based on category.
 - **(help | man | whatis | info | about) NAME** $\leftarrow$ displays help info about a given system command or user program specified in the value NAME.
 - **help-file** $\leftarrow$ display all available commands in the FILE I/O category.
 - **help-help** $\leftarrow$ display help on how to use the HELP facility.
 - **help-prog** $\leftarrow$ display all available commands in the PROGRAMMING category.
 - **help-ui** $\leftarrow$ Display all available commands in the USER INTERFACE category or sub-system.
 - **help-util** $\leftarrow$ display all available commands in the UTILITIES category.

After working on and testing the initial prototypes of the TEA OS, it was established that we shall need some means of controlling and configuring the user-interface mechanisms, and thus the following extra sub-system specification:

1. **INTERFACE Utilities:** access commands that can alter the user-experience and how certain user-interface aspects of the operating system work within TOS.
—[**Basic UI/UX Commands Interface**]:

- `uimini` ← Configure TEA OS interface to render using mini-terminal mode. Setting persists across logins until changed. This mode is suitable for environments such as using TOS on some Desktop Computer Systems.
- `uilog` ← Configure TEA OS interface to render using long/log-terminal mode. Setting persists across logins until changed. This mode is suitable for environments such as using TOS on the TEA MOBILE IDE.

And thus, the plan is now officially underway... to manifest an new operating system based on TEA!

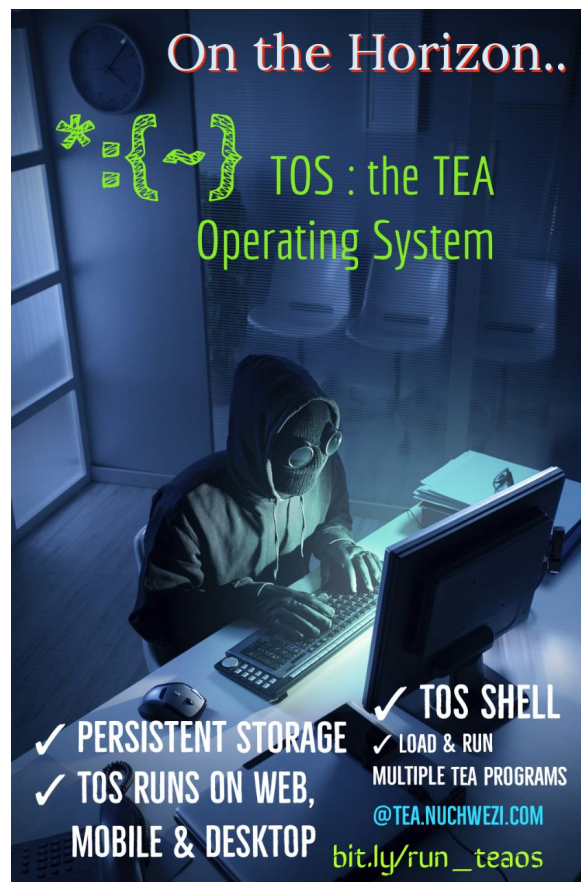
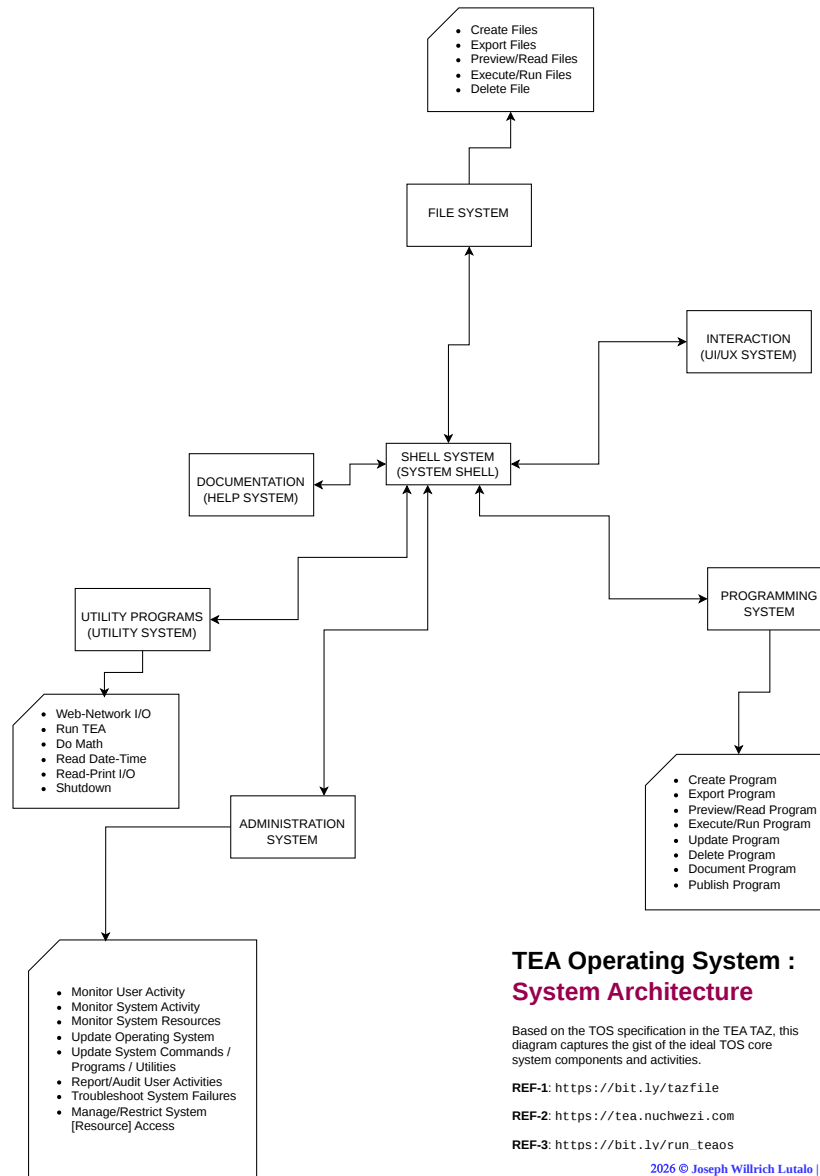


Figure 1: **A Hacker's Dream Come True?** The proposal to bring to life a **TEA Operating System!**

The link depicted in **Figure 1** being the first to have been officially shared with members of the TEA community concerning the original TEA OS pro-

posal, we had it point, not to the running instance of the TOS, but rather, to the potential staging source-file for the reference implementation of the operating system. And thus, those zealously interested in following what the latest working version of the single-file TEA OS code looks like — a program that one should be able to readily load into either the TEA WEB IDE or verify on any system with `tttt` package installed, and which they can then readily run, shall be found at the link: https://bit.ly/run_teaos

3 TEA OS Architecture



4 Contrasting TEA OS Architecture with Traditional Kernel-Centric Operating Systems

This section situates the TEA OS design presented in **Section 3** within the broader taxonomy of operating systems[5] and explains, in concrete technical terms, how TEA OS differs from a traditional kernel-centric implementation. The TEA OS was intentionally designed as a **minimal, portable runtime and service stack** — the TEA OS source (which we shall come to in), explicitly mentions the inventor’s intentions as:

Table 1: TEA OS’s inventor concerning the intended TEA OS core design

<i>a true general purpose, lightweight, minimal but powerful operating system that can essentially just fit inside a single string</i>
--

It should be worth noting that final aspect... “that fits inside a string”, because it shall without doubt explain why and how it is that the reference implementation of the TEA operating system is currently being shipped alongside other *standard* TEA programs[2] that essentially are loadable into the TEA WEB IDE via a mere JSON payload!

Further, note that it was originally intended that the OS should be able to run wherever the TEA runtime baseline is available (including directly in web browsers as already mentioned). That design goal drives a set of architectural tradeoffs that depart from the assumptions and responsibilities normally associated with an OS kernel.

4.1 High-level Summary of Intent

TEA OS is a compact, self-contained runtime (also what, in the terminology of Lutalo et al. we might technically refer to as a *software operating environment* [3]) that exposes five core capabilities: a **file system**, a **utility system**, a **help system**, a **programming system**, and a **UI system**. Interaction is primarily through a shell that supports the TEA language interpreter(s) and user-extensible commands. The project deliberately targets portability and minimalism: TEA OS assumes the presence of a TEA runtime rather than privileged hardware access, and it is implemented in a high-level language designed to be more compact and portable than mainstream scripting or programming languages [1]¹. As a consequence, TEA OS can execute in environments where traditional kernels cannot (for example, directly inside a web browser without a virtual machine).

¹Also refer to **Appendix A** on “Fundamentals of Programming via TEA” as laid down in [1].

4.2 Core Differences: Responsibilities and Assumptions

The following table summarizes the most relevant technical attributes and contrasts TEA OS against the canonical responsibilities of a traditional kernel. Each cell is a concise statement of whether the capability is typically a kernel responsibility and whether TEA OS implements it in its current form.

Attribute	Typical Kernel	TEA OS (current)
Hardware control and drivers	Direct device drivers; interrupt handling	Not implemented; relies on host runtime I/O
CPU scheduling and multitasking	Preemptive scheduling; context switching	Cooperative or single-threaded within TEA runtime
Memory management and protection	Virtual memory; isolation; MMU use	Managed by host runtime; no low-level protection
System call / syscall interface	Privileged syscall boundary	High-level API exposed by TEA runtime
Boot and initialization	Bootloader; kernel startup	TEA program initialization within host runtime
File system	Often kernel or kernel-assisted	Implemented as a TEA file subsystem (userland)
Security and privilege separation	Kernel enforces privileges	Relies on host sandboxing and TEA runtime
Extensibility (user commands)	Usually userland shells	Native: shell + TEA interpreters for extensions
Portability target	Hardware/ISA specific	Portable across TEA runtimes and browsers

Table 2: Checklist contrasting typical kernel responsibilities with TEA OS design.

4.3 Architectural Implications

Privilege and Trust Model. Traditional kernels operate in a privileged CPU mode and mediate access to hardware and shared resources [5, 6]. TEA OS, by contrast, is designed to run in an unprivileged environment provided by the TEA runtime [1]. This shifts the trust boundary: TEA OS trusts the host runtime (and the host environment, e.g., the browser) to provide isolation, I/O, and resource accounting. The practical implication is that TEA OS cannot, in its present form, enforce hardware-level isolation or implement mandatory

access control independent of the host.

Portability and Minimality. Because TEA OS targets a high-level runtime, it attains portability across platforms that implement the TEA baseline. This is a deliberate tradeoff: TEA OS sacrifices direct hardware control and low-level performance optimizations in exchange for extreme portability and compactness. The design goal of “fitting inside a string” and running without a VM is a distinguishing feature that enables deployment scenarios (e.g., in-browser shells, embedded documentation with runnable OS examples) that traditional kernels cannot match.

Service Placement and Layering. TEA OS places many services that would traditionally be split between kernel and userland entirely in the TEA userland layer (file system abstraction, utilities, help, programming environment, UI). This simplifies the kernel model (there is effectively no kernel in the traditional sense) but increases the responsibility of the TEA runtime and the host to provide secure, deterministic primitives.

4.4 When can TEA OS be called a Kernel?

The term *kernel* is meaningful only relative to a set of responsibilities [7]. If we wished to claim that TEA OS is a kernel, then we would be “conventionally” tasked to explicitly document or explain which of the canonical kernel responsibilities the TEA OS implements. Below is a practical checklist of items that, if implemented and documented, would justify calling TEA OS a **kernel** in the conventional sense:

- **Boot and initialization:** a documented boot sequence that initializes hardware or a hardware abstraction layer.
- **Device drivers and I/O:** drivers or a driver framework that directly manage devices or a well-specified hardware abstraction layer.
- **Interrupt handling:** mechanisms to receive and dispatch hardware interrupts or their equivalents.
- **Memory management:** support for memory allocation with protection boundaries (virtual memory or equivalent isolation).
- **Process scheduling:** preemptive multitasking or a scheduler that arbitrates CPU among isolated processes/threads.
- **System call interface:** a privileged boundary and syscall ABI for user programs to request kernel services.
- **Privilege separation and security:** enforcement of privilege levels and resource quotas independent of the host runtime.

- **Persistence and storage drivers:** direct control or well-specified management of persistent storage devices.

If TEA OS implements a substantial subset of these items (and documents the mechanisms and guarantees), it would then be reasonable to describe it as a kernel or as a kernel-like runtime [7]. However, since it currently does not, we shall currently be satisfied with the more accurate description of the TEA OS as a **minimal OS runtime and service stack**, as a “technology platform”, or as a userland **software operating environment** built on top of the TEA runtime.

5 TEA OS v1.1.0 source

```

1  #!/usr/bin/tttt -fc
2  I!:{ooooooooo-----
3  ---oo-----ooooo---ooooo--
4  ---oo-----oo-----o_oo---oo_
5  ---oo-----ooooooooo_oo---oo_
6  ---oo-----oo-----oo---oo_
7  ---oo-----ooooo---oooo_o_
8  -----}
9  x!:{
10 TEA OS v1.1.0 is Ready...}
11 v:vBanner | #i:
12
13 ###| The above is the "original" bootscreen
14 ###| The rest of the story follows..
15
16 *:{==| PREAMBLE |==}
17 L:lpREAMBLE_START
18
19 Y:vBanner #| V@:dvLOG
20 V@:dvLOG_start
21
22 #Of course, this is TEA programming
23 We can write verbatim documentation and
24 comments in the most natural of ways without
25 worry or concern. We sometimes shall
26 do just that.
27
28 V@:dvABOUT_TOS:{
29 #---[ABOUT TEA OS]
30
31 NAME: TEA Operating System
32 (also "TEA OS" or just "TOS")
33
34 ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>
35

```

```

36 SINCE: 9th APRIL, 2026
37
38 INTENT: To manifest a true general purpose, lightweight,
        minimal but powerful operating system that can
        essentially just fit inside a single string; to see what
        the limits of the mathematics of TRANSFORMATICS might be,
        and what we surely could accomplish with its proper use
        via modern portable computers.
39
40 ABOUT: TEA OS is implemented using the Transforming
        Executable Alphabet (TEA) programming language. It is
        based on a specification first laid out in the TEA TAZ.
        It is a project originating from UGANDA.
41
42 PROJECT HOME: Nuchwezi Research | https://tea.nuchwezi.com}
43
44 #--[GLOBAL CONSTANTS]
45 V@:dvOSName:{TEA OS}
46 V@:dvOSName_short:{TOS}
47 V@:dvOSVersion:{1.1.0}
48
49 V:vSLOGAN:{TEA OS! T can fit in a string!}
50 I!:{_____}|V:vDELIM|V@:dvDELIM
51 I!:{
52 =o==o==o==o==o==o==o==o=
53 }|V:vDELIM2|V@:dvDELIM2
54
55
56 #--[GLOBAL VARIABLES]
57 V:vUSERNAME:{user} # not used that much!
58
59
60 #--[DATABASE VARIABLES]
61 #Y@:dvLOG|i:{}|V@:dvLOG #all activity
62 V@:dvLastPS:{}
63 V@:dvLastCommand:{}
64 V@:dvLastOutput:{}
65
66 #TEA-DATABASE powered FILE-SYSTEM register
67 #{F1|F2|...|FN} s.t NAME(Fi)=ID(Fi)
68 Y@:dvFileList|I:{..}|V@:dvFileList
69
70 #TEA-DATABASE powered USER-PROGRAMS-SYSTEM register
71 #{F1|F2|...|FN} s.t NAME(Fi)=ID(Fi)
72 Y@:dvProgramsList|I:{run}|V@:dvProgramsList
73
74 #### INTERFACE CONFIG VARIABLE
75 # 0 forces original terminal mode: fit for TEA mobile IDE
76 # 1 is mini-terminal mode: fit for CLI and other WEB
77 # user can change this via commands:

```

```

78 # uimini -> 1
79 # uilog -> 0
80 # will default to mini if not already set
81 Y@:dvUIMODE_MINI|I:{1}|V@:dvUIMODE_MINI
82
83 #--[SOME FUNCTIONS]
84
85 #for presenting pretty message prompt..
86 # in vUIMODE_MINI=0
87 V:fPROMPT_DEFAULT:"
88 # FIRST: IMPORTANTLY...
89 v:vMSG #stores AI at invocation
90 y@:dvDELIM|v:vD2
91 y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
92 i!:{ } | g*:{
93 } :vD2:vMSG
94 v:vMSG
95 Y@:dfLogAI|v:f # a stealth import :)
96 y:vMSG|e*:f
97 #then show log+latest prompt
98 Y@:dvLOG
99 i: #returns input..
100 "
101
102 Y:fPROMPT_DEFAULT|V@:dfPROMPT_DEFAULT
103
104 #for presenting pretty message prompt..
105 # in vUIMODE_MINI=1
106 V:fPROMPT:"
107 # FIRST: IMPORTANTLY...
108 v:vMSG #stores AI at invocation
109
110 #check if we should instead be using default ui
111 y@:dvUIMODE_MINI | f!:^0$:lN_UIMODE_MINI
112 y@:dfPROMPT_DEFAULT|v:fPROMPT_DEFAULT
113 y:vMSG | E*:fPROMPT_DEFAULT
114 q!:# no need to proceed..
115 l:lN_UIMODE_MINI
116
117 #THEN imports...
118 y@:dfLastIO|v:fLastIO
119 y@:dfMakePS|v:fMakePS
120
121 y@:dvDELIM|v:vD2
122 y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
123 i!:{ } | g*:{
124 } :vD2:vMSG
125 v:vMSG
126 Y@:dfLogAI|v:f # a stealth import :)
127 y:vMSG|e*:f

```

```

128 #then show log+latest prompt
129 #Y@:dvLOG
130 E*:fMakePS|v:vNEXTPS
131 E*:fLastIO #fetch last I/O
132 x!:{
133 > }|#x*!:vNEXTPS
134 i: #returns input..
135 "
136
137 Y:fPROMPT|V@:dfPROMPT
138
139 #login function
140 V:fLogin:"
141 #import some functions from database...
142 y@:dfPROMPT_DEFAULT|v:fPROMPT
143 y@:dfLogAI|v:fLogAI
144 y@:dfLogO|v:fLogO
145 y@:dfGreetUser|v:fGreetUser
146
147 # show OS banner
148 y@:dvLOG_start | v:vSCREEN
149 e*:fLogAI
150
151 #greet user via stored name..
152 y!@:dvUSERNAME | f!:^0$:lGREET_USER
153 #Username not yet set, prompt and store
154 l:lSET_USERNAME
155 I!:{
156 Please set a Username:}|v:vPromptMSG
157 e*:fLogAI
158 y:vSCREEN | x*!:vPromptMSG | i:
159 t!.:|v:|v!.:|f!:^0$:lSET_USERNAME
160 y:|V@:dvUSERNAME # store in database
161 e*:fLogAI
162
163 l:lGREET_USER
164 e*:fGreetUser|v:vGreetings
165 e*:fLogO | e*:fPROMPT
166 "
167
168 #load username and greet
169 V:fGreetUser:"
170 y@:dvUSERNAME
171 x:{
172 Hello, }|x!:{. Welcome to TOS!}
173 "
174
175 Y:fGreetUser|V@:dfGreetUser
176
177 #for os name..

```



```

178 V:fOSHandle:"
179 y@:dvOSName | v:vN
180 y@:dvOSVersion | v:vV
181 g*:{ v}:vN:vV
182 "
183
184 Y:fOSHandle|V@:dfOSHandle
185
186 #for default prompt..
187 V:fMakePS:"
188 z.:TIME |v:vT
189 y@:dvOSName_short | v:vPWD
190 v:vD:{>}
191 g*:{/}:vPWD:vT:vD
192 "
193
194 Y:fMakePS|V@:dfMakePS
195
196 #log last cmd+0
197 V:fLogIO:"
198 #load+execute function from database!
199 y@:dfLastIO|e:
200 v:vLastIO
201 y@:dvLOG | v:vLOG
202 g*:{
203 }:vLOG:vLastIO | v@:dvLOG
204 "
205
206 #suffix AI to log
207 V:fLogAI:"
208 v:vAI
209 y@:dvLOG | v:vLOG
210 g*:{
211 }:vLOG:vAI | v@:dvLOG
212 #return AI
213 y:vAI
214 "
215
216 #also make it a db function
217 Y:fLogAI|v@:dfLogAI
218
219 #store AI as last-0..
220 V:fLog0:"
221 v@:dvLastOutput
222 v@:dvDATABUFFER #also stash into special data buffer
223 Y@:dvLastOutput
224 "
225
226 #fetch last I/O..
227 V:fLastIO:"

```

```

228 y@:dvLastPS | v:vPS
229 y@:dvLastCommand | v:vC
230 y@:dvLastOutput | v:v0
231 g*:{}:vPS:vC|v:vCC
232 g*:{
233 }:vCC:v0
234 "
235 #store that function in the database too (so we can later
    use it in context-unaware functions).
236 y:fLastIO|v@:dfLastIO
237
238 L:1PREAMBLE_END
239 #:{=====}
240
241 *:{==|UTILITY SYSTEM |==}
242 L:1SYS_UTILITY_START
243
244 #--[UTILITY Functions]
245
246 #fix URL
247 V:fFixURL:"
248 v:vURL|z:
249 f:{^https?://}:1FINE
250 y:vURL|x:{https://}
251 v:vURL
252 l:1FINE
253 y:vURL
254 "
255
256 #reset+clear log
257 V:fResetLog:"
258 y@:dvLastOutput | v@:dvLastOutput_backup
259 c!@:dvLOG
260 c*:vLOG
261 C!@:dvLastOutput
262 y@:dfGreetUser|E:|v:vGreetings
263 y@:dvLOG_start
264 x*!:vGreetings
265 "
266
267 # make available for [db] import..
268 Y:fResetLog|V@:dfResetLog
269
270
271 V:fGetLatestOutputBuffer:"
272 y@:dvLastOutput_backup
273 f!:`~$:1N_USED_BACKUP
274 y@:dvLastOutput
275 l:1N_USED_BACKUP
276 c@:dvLastOutput_backup

```

```

277 "
278
279 Y:fGetLatestOutputBuffer|V@:dfGetLatestOutputBuffer
280
281 L:1SYS_UTILITY_END
282 #:{=====}
283
284 *:{==|HELP SYSTEM |==}
285 L:1SYS_HELP_START
286
287 #--[HELP/DOC Functions]
288
289 #present USER INTERFACE help-category commands
290 V:fGetUIMenu:"
291 y@:dfOSHandle|e:|x!:{ INTERFACE System Commands:
292 uimini
293 uilog}
294 v:vMSG
295 y@:dvDELIM2|v:vD
296 g*:{}:vD:vMSG:vD
297 "
298
299 #present FILE help-category commands
300 V:fGetFileCMDMenu:"
301 y@:dfOSHandle|e:|x!:{ FILE System Commands:
302 create FILENAME
303 init FILENAME
304 write FILENAME
305 (rename | rn | mv | move) FILENAME NEWNAME
306 (copy|clone) FILE1 FILE2
307 exists FILENAME
308 find PATTERN
309 (show | cat | read) FILENAME
310 export FILENAME
311 (del | delete | rm) FILENAME
312 (ls | list | dir | files) PATTERN}
313 v:vMSG
314 y@:dvDELIM2|v:vD
315 g*:{}:vD:vMSG:vD
316 "
317
318 #present PROGRAMMING help-category commands
319 V:fGetProgrammingMenu:"
320 y@:dfOSHandle|e:|x!:{ PROGRAMMING System Commands:
321 (runnable | register | add-program) FILENAME
322 (isrunnable | canrun) FILENAME
323 (run | exec | execute) FILENAME
324 (del-prog | delete-program | unregister) FILENAME
325 (update | update-program) FILENAME
326 PROGRAM

```

```

327 (help | man | whatis | info | about) PROGRAM}
328 v:vMSG
329 y@:dvDELIM2|v:vD
330 g*:{}:vD:vMSG:vD
331 "
332
333 #present UTILITIES help-category commands
334 V:fGetUtilityMenu:"
335 y@:dfOSHandle|e:|x!:{ UTILITY System Commands:
336 clear | reset
337 date
338 echo MESSAGE
339 exit
340 export
341 (fetch | web) [URL] [FILENAME]
342 logout
343 math EXPRESSION
344 print
345 tea CODE
346 time
347 timestamp
348 username NAME
349 whoami | name}
350 v:vMSG
351 y@:dvDELIM2|v:vD
352 g*:{}:vD:vMSG:vD
353 "
354
355 #present main help-category commands
356 V:fGetHelpMenu:"
357 y@:dfOSHandle|e:|x!:{ HELP System Commands:
358 about
359 (help | man | whatis | info | about) [NAME]
360 help-file
361 help-help
362 help-prog
363 help-ui
364 help-util}
365 v:vMSG
366 y@:dvDELIM2|v:vD
367 g*:{}:vD:vMSG:vD
368 "
369
370 #present main about os info
371 V:fGetAboutTOS:"
372 y@:dfOSHandle|e:|x:{ABOUT: }
373 v:vMSG
374 y@:dvABOUT_TOS|x*:vMSG|v:vMSG
375 y@:dvDELIM2|v:vD
376 g*:{}:vD:vMSG:vD

```

```

377 "
378
379 #present help-on a given topic
380 V:fGetHelpOn:"
381 #asumes topic is passed via AI
382 v:vHelpTOPIC
383
384 #some imports...
385 y@:dfGetFilenameProtocolMSG|v:fGetFilenameProtocolMSG
386 E*:fGetFilenameProtocolMSG | v:vFILENAME_PROTOCOL
387
388 #set default..
389 r@:{That help topic [#vHelpTOPIC#] is not understood or
    supported. Try the 'help' command to see what help
    commands there are. For example 'help whatis'}|v:vMSG
390
391 y:vHelpTOPIC
392
393 #> help about
394 f!:^about$:lN_h0
395 v:vMSG:{about <~ Displays basic info about the currently
    running version of the TEA Operating System.}
396 j:lh1_present
397 l:lN_h0
398
399 #> help whatis
400 f!:^whatis$:lN_h1
401 v:vMSG:{whatis NAME <~ Displays help info about a given
    system command or user program.}
402 j:lh1_present
403 l:lN_h1
404
405 #> help man
406 f!:^man$:lN_h1b
407 v:vMSG:{man NAME <~ Displays help info about a given system
    command or user program. It is just another alias for '
    whatis' or 'help'}
408 j:lh1_present
409 l:lN_h1b
410
411 #> help help
412 f!:^help$:lN_h2
413 v:vMSG:{Present menu for user to access help about all
    available [essential] commands based on category.}
414 j:lh1_present
415 l:lN_h2
416
417 f!:^help-file$:lN_h3
418 v:vMSG:{Display all available commands in the FILE I/O
    category or sub-system.}

```

```

419 j:lh1_present
420 l:1N_h3
421
422 f!:^help-prog$:1N_h4
423 v:vMSG:{Display all available commands in the PROGRAMMING
      category or sub-system.}
424 j:lh1_present
425 l:1N_h4
426
427 f!:^help-util$:1N_h5
428 v:vMSG:{Display all available commands in the UTILITIES
      category or sub-system.}
429 j:lh1_present
430 l:1N_h5
431
432 f!:^help-help$:1N_h6
433 v:vMSG:{Display help on how to use the HELP facility or sub-
      system.}
434 j:lh1_present
435 l:1N_h6
436
437 f!:{^(clear|reset)}$:1N_hu0
438 v:vCMD
439 r@:{#vCMD# <~ Resets and clears the current TEA OS session
      log since login.} | v:vMSG
440 j:lh1_present
441 l:1N_hu0
442
443 f!:{^(fetch|web)}$:1N_hu1
444 v:vCMD
445 r@:{#vCMD# URL <~ fetch and output contents of the specified
      [online/network] resource.
446
447 #vCMD# URL FILENAME <~ fetch and save contents at URL into
      file record named FILENAME. #vFILENAME_PROTOCOL#} | v:
      vMSG
448 j:lh1_present
449 l:1N_hu1
450
451 f!:^tea$:1N_hu2
452 v:vMSG:{tea CODE.. <~ take everything following the keyword
      'tea' as a TEA program, run and output the result: It's a
      simple way to write and run basic TEA programs (TEA-one-
      liners) in TOS.}
453 j:lh1_present
454 l:1N_hu2
455
456 f!:^math$:1N_hu3
457 v:vMSG:{math EXPRESSION <~ execute basic mathematics on the
      command-line as specified in EXPRESSION.}

```

```
458 j:lh1_present
459 l:1N_hu3
460
461 f!:^date$:1N_hu4
462 v:vMSG:{date <~ Show the current date.}
463 j:lh1_present
464 l:1N_hu4
465
466 f!:^time$:1N_hu5
467 v:vMSG:{time <~ Show the current time.}
468 j:lh1_present
469 l:1N_hu5
470
471 f!:^timestamp$:1N_hu6
472 v:vMSG:{timestamp <~ Show the current timestamp.}
473 j:lh1_present
474 l:1N_hu6
475
476 f!:^print$:1N_hu7
477 v:vMSG:{print <~ Show or preview current output buffer
      contents (usually, output of last command). After
      clearing log, but also after exit/system shutdown/login
      also shows final output before the log was reset.}
478 j:lh1_present
479 l:1N_hu7
480
481 f!:^export$:1N_hu8
482 r@:{export <~ Take the output of the last command or rather,
      current contents of the output buffer and export that
      outside the operating system (also terminates the
      operating system session).
483
484 export FILENAME <~ Exit the system, outputting the contents
      of given file. #vFILENAME_PROTOCOL#} | v:vMSG
485 j:lh1_present
486 l:1N_hu8
487
488 f!:^exit$:1N_hu9
489 v:vMSG:{exit <~ Ends the TOS session returning nothing.}
490 j:lh1_present
491 l:1N_hu9
492
493 f!:^logout$:1N_hu10
494 v:vMSG:{logout <~ Resets username and system logs and ends
      the TOS session returning nothing.}
495 j:lh1_present
496 l:1N_hu10
497
498 f!:^username$:1N_hu11
```

```

499 v:vMSG:{username NAME <~ Overrides current username with
    specified value.}
500 j:lh1_present
501 l:1N_hu11
502
503 f!:^help-ui$:1N_h12
504 v:vMSG:{Display all available commands in the USER INTERFACE
    category or sub-system.}
505 j:lh1_present
506 l:1N_h12
507
508 f!:^uimini$:1N_h13
509 v:vMSG:{uimini <~ Configure TEA OS interface to render using
    mini-terminal mode. Setting persists across logins until
    changed. This mode is suitable for environments such as
    using TOS on some Desktop Computer Systems.}
510 j:lh1_present
511 l:1N_h13
512
513 f!:^uilog$:1N_h14
514 v:vMSG:{uilog <~ Configure TEA OS interface to render using
    long/log-terminal mode. Setting persists across logins
    until changed. This mode is suitable for environments
    such as using TOS on the TEA MOBILE IDE.}
515 j:lh1_present
516 l:1N_h14
517
518 f!:^whoami$:1N_h15
519 v:vCMD
520 r@:{#vCMD# <~ Display the NAME of currently active USER.} |
    v:vMSG
521 j:lh1_present
522 l:1N_h15
523
524 f!:^create$:1N_h16
525 r@:{create FILENAME <~ Creates blank file with name FILENAME
    . #vFILENAME_PROTOCOL#}|v:vMSG
526 j:lh1_present
527 l:1N_h16
528
529 f!:^{(ls|list|dir|files)}$:1N_h17
530 v:vCMD
531 r@:{#vCMD# <~ Display list of ALL available files.
532
533 #vCMD# PATTERN <~ Display list of ONLY files matching
    regular expression PATTERN.}
534 | v:vMSG
535 j:lh1_present
536 l:1N_h17
537

```



```

538 f!:^init$:1N_h18
539 r@:{init FILENAME <~ Create a file using the given name and
    the currently available input/last-output (or the PRINT
    buffer) or blank if none. #vFILENAME_PROTOCOL#} | v:vMSG
540 j:lh1_present
541 l:1N_h18
542
543 f!:^write$:1N_h19
544 r@:{write FILENAME <~ Write the contents of the current
    available input/last-output (or the PRINT buffer) or
    blank if none, to a file with the specified name. If the
    file already exists, essentially overwrites its contents.
    #vFILENAME_PROTOCOL#} | v:vMSG
545 j:lh1_present
546 l:1N_h19
547
548 f!:{^(show|cat|read)}$:1N_h20
549 v:vCMD
550 r@:{#vCMD# FILENAME <~ Preview the contents of given file. #
    vFILENAME_PROTOCOL#} | v:vMSG
551 j:lh1_present
552 l:1N_h20
553
554 f!:^echo$:1N_h21
555 v:vMSG:{echo MESSAGE <~ Simply print MESSAGE as is.}
556 j:lh1_present
557 l:1N_h21
558
559 f!:{^(find)}$:1N_h22
560 v:vCMD
561 r@:{#vCMD# PATTERN <~ Display list of ONLY files matching
    regular expression PATTERN.}
562 | v:vMSG
563 j:lh1_present
564 l:1N_h22
565
566 f!:{^(del|delete|rm)}$:1N_h23
567 v:vCMD
568 r@:{#vCMD# PATTERN <~ Remove from storage, any and all files
    whose name matches PATTERN. CAN NOT DELETE registered
    user program files though.} | v:vMSG
569 j:lh1_present
570 l:1N_h23
571
572 f!:{^(rename|rn|mv|move)}$:1N_h24
573 v:vCMD
574 r@:{#vCMD# FILENAME NEWNAME <~ Change the name of an
    existing file from FILENAME to NEWNAME. WILL ALSO update
    the registered [user] programs list if FILENAME was a
    registered program. #vFILENAME_PROTOCOL#} | v:vMSG

```

```

575 j:lh1_present
576 l:1N_h24
577
578 f!:{^(exists)$}:1N_h25
579 v:vCMD
580 r@:{#vCMD# FILENAME <~ Show whether or not a file named
    FILENAME exists. #vFILENAME_PROTOCOL#} | v:vMSG
581 j:lh1_present
582 l:1N_h25
583
584 f!:{^(copy|clone)$}:1N_h26
585 v:vCMD
586 r@:{#vCMD# FILENAME FILE2 <~ Copies contents of FILENAME to
    FILE2 creating the destination if it doesn't exist. #
    vFILENAME_PROTOCOL#} | v:vMSG
587 j:lh1_present
588 l:1N_h26
589
590 f!:{^(runnable|register|add-program)$}:1N_h27
591 v:vCMD
592 r@:{#vCMD# FILENAME <~ Recognize FILENAME object as an
    executable [user] program or command. FILENAME must
    already be an existing TEA file. #vFILENAME_PROTOCOL#} |
    v:vMSG
593 j:lh1_present
594 l:1N_h27
595
596 f!:{^programs$}:1N_h28
597 v:vCMD
598 r@:{#vCMD# <~ Show list of all available registered [TEA]
    user programs by their registered names.
599
600 #vCMD# PATTERN <~ With PATTERN specified, ONLY show those
    programs whose names match regular expression PATTERN.}
601 | v:vMSG
602 j:lh1_present
603 l:1N_h28
604
605 f!:{^(run|exec|execute)$}:1N_h29
606 v:vCMD
607 r@:{#vCMD# FILENAME <~ Execute the given file record as a [
    TEA] program. #vFILENAME_PROTOCOL#} | v:vMSG
608 j:lh1_present
609 l:1N_h29
610
611 f!:{^(isrunnable|canrun)$}:1N_h30
612 v:vCMD
613 r@:{#vCMD# FILENAME <~ Show whether or not a file named
    FILENAME can be executed as a [TEA] program. #
    vFILENAME_PROTOCOL#} | v:vMSG

```

```

614 j:lh1_present
615 l:1N_h30
616
617 f!:{^(del-prog|delete-program|unregister)$}:1N_h31
618 v:vCMD
619 r@:{#vCMD# FILENAME <~ Revokes status of FILENAME as an
    executable user program (doesn't actually delete the
    underlying file object though). #vFILENAME_PROTOCOL#} | v
    :vMSG
620 j:lh1_present
621 l:1N_h31
622
623
624 f!:{^(update|update-program)$}:1N_h32
625 v:vCMD
626 r@:{#vCMD# PROGRAMFILE FILENAME <~ Replace/override original
    program named PROGRAMFILE with contents of FILENAME (
    updates a program's code [in PROGRAMFILE] preserving its
    existing name). Both files PROGRAMFILE and FILENAME must
    actually exist for this to work, but only PROGRAMFILE
    needs to be a registered program. #vFILENAME_PROTOCOL#} |
    v:vMSG
627 j:lh1_present
628 l:1N_h32
629
630 f!:{^(program)$}:1N_h33
631 v:vCMD | z!: | v:vCMD #use upercase..
632 r@:{#vCMD# <~ Executes #vCMD# as a system command if it
    exists --- first, if it is any of the standard commands
    in TEA OS, otherwise if it is a valid filename "#vCMD#"
    that is already recognized/registered as a user-created
    program - this later case similar to calling "run #vCMD
    #", otherwise shows error message command-not-found.} | v
    :vMSG
633 j:lh1_present
634 l:1N_h33
635
636 # the user-program catch-all...
637 f!:{^([a-z0-9_]+(\.[a-z0-9]+)?)$}:1N_h34
638 v:vPROGRAMFILE
639 y@:dfProgramExists | v:fProgramExists #import
640 y:vPROGRAMFILE | e*:fProgramExists
641 f!:{^PROGRAM EXISTS$}:1N_h34
642 r@:{#vPROGRAMFILE# <~ Is a valid user-registered [TEA]
    program. You might run or execute it by directly invoking
    "#vPROGRAMFILE#" or via "run #vPROGRAMFILE#".} | v:vMSG
643 j:lh1_present
644 l:1N_h34
645
646 l:lh1_present

```

```

647 y@:dvDELIM2|v:vD
648 g*:{}:vD:vMSG:vD
649 "
650
651 L:1SYS_HELP_END
652 #:{=====}
653
654 *:{==|FILE SYSTEM|==}
655 L:1SYS_FILE_START
656
657 #--[FILE SYSTEM Functions]
658
659 #f(N)->DFS_N
660 V:fGetFilenameProtocolMSG:"i!:{Valid FILENAME should consist
        of ONLY letters a-zA-Z, numbers 0-9, underscore _ and/or
        a dot .}"
661 Y:fGetFilenameProtocolMSG|V@:dfGetFilenameProtocolMSG
662
663 #f(N)->DFS_N
664 V:fGenDBFilename:"x:{DFS_}"
665 Y:fGenDBFilename|V@:dfGenDBFilename
666
667 #f(DFS_N)->N
668 V:fDecodeDBFilename:"d:~DFS_"
669 Y:fDecodeDBFilename|V@:dfDecodeDBFilename
670
671 #f(N)->0|1
672 V:fIsFileNameValid:"
673 f!:^[a-z0-9_]+(\.[a-z0-9]+)?$:1BAD_FILE_NAME
674 i!:1|q!:
675 l:1BAD_FILE_NAME
676 i!:0
677 "
678 Y:fIsFileNameValid|V@:dfIsFileNameValid
679
680
681 #create FILENAME
682 V:fCreateFile:"
683 # first, grab incoming function argument...
684 v:vFILENAME|t!.:|v:vFNAME
685
686 #then import functions..
687 y@:dfGenDBFilename|v:fGenDBFilename
688 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
689 y@:dfIsFileNameValid|v:fIsFileNameValid
690
691 y:vFNAME | f:~$:1NFINE # filename shouldn't be blank
692 # file name should conform to accepted patterns
693 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
694 r@:{Invalid File Name [#vFNAME#]}!

```

```

695 FILE NOT CREATED.}|q!:
696 l:lGOOD_FILENAME
697
698 # fetch current file name list..
699 y@:dvFileList | v:vFileList
700 # ensure file name doesn't yet exist in records
701 #e.g v:vFileList:{F1|F2|F33|File|File}
702 #v:vFNAME:{f1}
703 # check if file name is unique
704 r@:{(^#vFNAME#\)|(\|#vFNAME#\)|(\|#vFNAME#\$)}|v:
    vFNAME_REGEX
705 y:vFileList | f*!:vFNAME_REGEX:lFNAME_UNIQUE:
    lFNAME_DUPLICATE
706 l:lFNAME_UNIQUE
707 # add name to file list
708 g*:{|}:vFileList:vFNAME | v:vFileList | v@:dvFileList
709 #also create blank file record in db filesystem
710 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
711
712 v@:dvDATA:{} # we'll write empty file
713 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
714
715 i!:{CREATED NEW BLANK FILE: }|x*!:vFNAME
716 q!:
717 l:lFNAME_DUPLICATE
718 i!:{File Already Exists!
719 NOT DUPLICATING FILE: }|x*!:vFNAME
720 q!:
721 l:lNFINE
722 i!:{Invalid [blank] File Name!
723 FILE NOT CREATED.}
724 "
725
726 Y:fCreateFile|V@:dfCreateFile
727
728
729 #delete FILENAME
730 V:fDeleteFile:"
731 # first, grab incoming function argument...
732 v:vFILENAME|t!..|v:vFNAME
733
734 #then import functions..
735 y@:dfGenDBFilename|v:fGenDBFilename
736 y@:dfIsFileNameValid|v:fIsFileNameValid
737
738 y:vFNAME | f:~$:lNFINE # filename shouldn't be blank
739 # file name should conform to accepted patterns
740 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
741 r@:{Invalid File Name [#vFNAME#]!
742 FILE NOT DELETED.}|q!:

```

```

743 l:1GOOD_FILENAME
744
745 # fetch current file name list..
746 y@:dvFileList | v:vFileList
747 # remove file name from records
748
749 #v:vFileList:{F1|F2|F33|F3|File|File}
750 #v:vFNAME:{File}
751 # get filename pattern
752
753 # create delete filename patterns + apply them..
754 r@:{(^#vFNAME#\)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
755 r@:{(\|#vFNAME#\)|}|v:vFNAME_REGEX_MID
756 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
757 # use it to del
758 y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
759 y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
760 v:vDIV:{|}
761 r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList
762
763 # update file list
764 y:vFileList | v@:dvFileList
765 #also delete file record from db filesystem
766 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
767 r@:{c!@:#vdFNAME#}|E:
768
769 i!:{DELETED FILE: }|x*!:vFNAME
770 q!:
771
772 l:1NFINE
773 i!:{Invalid [blank] File Name!
774 FILE NOT DELETED.}
775 "
776
777 Y:fDeleteFile|V@:dfDeleteFile
778
779 #ls | list | dir | files
780 V:fShowFilesList:"
781 # takes no parameters ;)
782 # fetch current file name list..
783 y@:dvFileList
784 #process and present..
785 h!:{\|}|d:{\|}
786 "
787
788 #(ls | list | dir | files | find) PATTERN
789 V:fSearchFilesList:"
790 # takes file regex pattern parameter
791 v:vREGEX_PATTERN
792 # fetch current file name list..

```

```

793 y@:dvFileList
794 #process and present..
795 h!:{\}|d:{\}|
796 # only keep what matches...
797 k*:vREGEX_PATTERN
798 "
799
800 #(del | delete | rm) PATTERN
801 V:fDeleteFilesList:"
802 # takes file regex pattern parameter
803 v:vREGEX_PATTERN
804
805 #imports..
806 y@:dfDeleteFile | v:fDeleteFile
807 y@:dfProgramExists | v:fProgramExists
808
809 # fetch current file name list..
810 y@:dvFileList
811 #process and present..
812 h!:{\}|d:{\}|
813 # only keep what matches...
814 k*:vREGEX_PATTERN
815 g:{|} | v:vDelFileList | v:vDelFileListORIG
816 f:~$:NO_FILES_TO_DELETE
817 # to track files we couldn't delete
818 v:vNOTDelFileList:{}
819
820 # go through files and delete..
821 v:vNCOUNTER:0
822 v:vNCOUNTER_DEL:0
823
824 l:lSTART_DEL
825 y:vDelFileList
826 f:~$:lDONE_DEL
827 #fetch..
828 y:vDelFileList | d!:{^[^\|]+\|?}
829 d:{\}| | v:vDelFileName
830
831 #if actually blank.. go pop
832 f:~$:lPOPDEL_LIST
833
834 # ensure it is not a registered program
835 y:vDelFileName | e*:fProgramExists | f:{~PROGRAM EXISTS$}:
      lDELETE_FAILED
836
837 #r@:{--Continuing to delete file [#vDelFileName#]} | q!:
838
839 # delete file..
840 y:vDelFileName | e*:fDeleteFile
841 #q!: # check...

```

```

842
843 f!:{DELETED FILE}:lDELETE_FAILED:lDELETED_OK
844 l:lDELETE_FAILED
845 # add name to NOT deleted file list
846 g*:{|}:vNOTDelFileList:vDelFileName | v:vNOTDelFileList
847 j:ldONE_DELETED_OK
848 l:lDELETED_OK
849 y:vNCOUNTER_DEL | x!:+1 | r.: | v:vNCOUNTER_DEL
850 l:ldONE_DELETED_OK
851 y:vNCOUNTER | x!:+1 | r.: | v:vNCOUNTER
852
853 #pop..
854 l:lPOPDEL_LIST
855 y:vDelFileList | d:{^[^\|]+\|?} | v:vDelFileList
856 #x:{Remaining: } | q!:
857 j:lSTART_DEL # iterate
858
859 l:ldONE_DEL
860 y:vDelFileListORIG|d:{^[^\|]:{\\|}$}|v:vDelFileListORIG
861 y:vNOTDelFileList|d:{^[^\|]:{\\|}$}|v:vNOTDelFileList
862 r@:{#vNCOUNTER# - #vNCOUNTER_DEL#}|r.:|v:vNCOUNTER_NDEL
863 y:vNCOUNTER_DEL | f:~1$:lDEL_REP_1
864 r@:{DELETED #vNCOUNTER_DEL# file[s] from list:
865 #vDelFileListORIG#
866 EXCEPT #vNCOUNTER_NDEL#:
867 #vNOTDelFileList#}|q!:
868
869 l:ldEL_REP_1
870 y:vDelFileListORIG|d:{^[^\|]:{\\|}$}|v:vDelFileListORIG
871 y:vNCOUNTER | f:~1$:lDEL_REP_2
872 r@:{DELETED ONLY #vNCOUNTER_DEL# file:
873 #vDelFileListORIG#
874 }|q!:
875
876 l:ldEL_REP_2
877 y:vDelFileListORIG|d:{^[^\|]:{\\|}$}|v:vDelFileListORIG
878 y:vNOTDelFileList|d:{^[^\|]:{\\|}$}|v:vNOTDelFileList
879 r@:{DELETED ONLY #vNCOUNTER_DEL# file from list:
880 #vDelFileListORIG#
881 EXCEPT #vNCOUNTER_NDEL#:
882 #vNOTDelFileList#}|q!:
883
884 l:NO_FILES_TO_DELETE
885 r@:{#vREGEX_PATTERN# matched no FILES!
886 NO FILE WAS DELETED}
887 "
888
889 #init FILENAME
890 V:fInitFile:"
891 v:vFILENAME|t!.:|v:vFNAME

```



```

892
893 #imports...
894 y@:dfCreateFile|v:fCreateFile
895 y@:dfGenDBFilename|v:fGenDBFilename
896 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
897 y@:dfIsFileNameValid|v:fIsFileNameValid
898
899 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
900 r@:{Invalid File Name [#vFNAME#]!
901 FILE NOT CREATED.}|q!:
902 l:lGOOD_FILENAME
903
904 # then, try to create the file...
905 y:vFNAME | E*:fCreateFile
906 # check if it was created...
907 f!:{CREATED NEW BLANK FILE}:lFILE_NOT_CREATED
908 # get name of that newly created file...
909 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
910
911 # get current output buffer
912 y@:dvDATABUFFER | v@:dvDATA
913
914 # only if it was empty then...
915 f!:^$:lFINE_DATA_BUFFER
916 # use print buffer...
917 e*:fGetLatestOutputBuffer | v@:dvDATA
918 l:lFINE_DATA_BUFFER
919
920 # store it into file system record...
921 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
922
923 #a test...
924 #r@:{y@:#vdFNAME#}|E:
925 #x:{File now contains:
926 #}|q!:
927
928 i!:{CREATED and INITIALIZED NEW File: }|x*!:vFNAME
929 q!:
930 l:lFILE_NOT_CREATED
931 i!:{FAILED TO CREATE FILE: } | x*!:vFNAME
932 "
933
934 #write FILENAME
935 V:fWriteFile:"
936 v:vFILENAME|t!.:|v:vFNAME
937
938 #imports...
939 y@:dfCreateFile|v:fCreateFile
940 y@:dfGenDBFilename|v:fGenDBFilename
941 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer

```

```

942 y@:dfIsFileNameValid|v:fIsFileNameValid
943
944 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
945 r@:{Invalid File Name [#vFNAME#]!
946 FILE NOT WRITTEN.}|q!:
947 l:1GOOD_FILENAME
948
949 # then, try to create the file...
950 y:vFNAME | E*:fCreateFile
951 # check if it was created...
952 f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:
    lFILE_NOT_CREATED
953 # get name of that file...
954 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
955 # get current output buffer
956 y@:dvDATABUFFER | v@:dvDATA
957
958 # only if it was empty then...
959 f!:^$:lFINE_DATA_BUFFER
960 # use print buffer...
961 e*:fGetLatestOutputBuffer | v@:dvDATA
962 l:1FINE_DATA_BUFFER
963
964 # store it into file system record...
965 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
966
967 i!:{DATA WRITTEN to File: }|x*!:vFNAME
968 q!:
969 l:1FILE_NOT_CREATED
970 i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
971 "
972
973 #savedata FILENAME
974 #reads special buffer v@:dvDATABUFFER
975 V:fSaveDataToFile:"
976 v:vFILENAME|t!.:|v:vFNAME
977
978 #imports...
979 y@:dfCreateFile|v:fCreateFile
980 y@:dfGenDBFilename|v:fGenDBFilename
981 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
982 y@:dfIsFileNameValid|v:fIsFileNameValid
983
984 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
985 r@:{Invalid File Name [#vFNAME#]!
986 FILE NOT WRITTEN.}|q!:
987 l:1GOOD_FILENAME
988
989 # then, try to create the file...
990 y:vFNAME | E*:fCreateFile

```

```

991 # check if it was created...
992 f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:
    lFILE_NOT_CREATED
993 # get name of that file...
994 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
995 # get current data buffer
996 # and store it into file system record...
997 r@:{y@:dvATABUFFER|v@:#vdFNAME#}|E:
998
999 i!:{DATA WRITTEN to File: }|x*!:vFNAME
1000 q!:
1001 l:lFILE_NOT_CREATED
1002 i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
1003 "
1004
1005 #read FILENAME
1006 V:fReadFile:"
1007 v:vFILENAME|t!..:|v:vFNAME
1008
1009 #imports...
1010 y@:dfGenDBFilename|v:fGenDBFilename
1011 y@:dfIsFileNameValid|v:fIsFileNameValid
1012
1013 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1014 r@:{Invalid File Name [#vFNAME#]!
1015 FILE NOT READ.}|q!:
1016 l:lGOOD_FILENAME
1017
1018 # then, try to read the file...
1019 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
1020
1021 r@:{y@:#vdFNAME#}|E:
1022 "
1023
1024 y:fReadFile | v@:dfReadFile
1025
1026 #exists FILENAME
1027 V:fFileExists:"
1028 # first, grab incoming function argument...
1029 v:vFILENAME|t!..:|v:vFNAME
1030
1031 #then import functions..
1032 y@:dfGenDBFilename|v:fGenDBFilename
1033 y@:dfIsFileNameValid|v:fIsFileNameValid
1034
1035 y:vFNAME | f:~$:lNFINE
1036 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1037 r@:{Invalid File Name [#vFNAME#]!
1038 FILE MAY NOT EXIST.}|q!:
1039 l:lGOOD_FILENAME

```

```

1040
1041 # fetch current file name list..
1042 y@:dvFileList | v:vFileList
1043
1044
1045 #v:vFileList:{F1|F2|F33|F3|File|File}
1046 #v:vFNAME:{f1}
1047
1048 # create filename patterns..
1049 r@:{(^#vFNAME#\)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
1050 r@:{(\|#vFNAME#\)|}|v:vFNAME_REGEX_MID
1051 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
1052 # use them to test for existence
1053 y:vFileList | f*:vFNAME_REGEX_ENDS:lFILE_EXISTS
1054 f*:vFNAME_REGEX_ALL:lFILE_EXISTS
1055 f*:vFNAME_REGEX_MID:lFILE_EXISTS
1056
1057 i!:{FILE DOES NOT EXIST}|q!:
1058
1059 l:lFILE_EXISTS
1060 i!:{FILE EXISTS}
1061 q!:
1062
1063 l:lNFINE
1064 i!:{Invalid [blank] File Name!
1065 FILE DOES NOT EXIST.}
1066 "
1067
1068 L:lSYS_FILE_END
1069 #:{=====}
1070
1071
1072 *:{==|PROGRAMMING SYSTEM |==}
1073 L:lSYS_PROG_START
1074
1075 #--[PROGRAMMING SYSTEM Functions]
1076
1077
1078 #runnable FILENAME
1079 V:fRegisterProgramFile:"
1080 # we assume given program file already exists!
1081 # first, grab incoming function argument...
1082 v:vFILENAME|t!..|v:vFNAME
1083
1084 #then import functions..
1085 y@:dfGenDBFilename|v:fGenDBFilename
1086 y@:dfIsFileNameValid|v:fIsFileNameValid
1087
1088 y:vFNAME | f:~$:lNFINE
1089 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME

```

```

1090 r@:{Invalid File Name [#vFNAME#]!
1091 CANNOT REGISTER PROGRAM.}|q!:
1092 l:lGOOD_FILENAME
1093
1094 # fetch current program file name list..
1095 y@:dvProgramsList | v:vProgramsList
1096
1097 #v:vFileList:{F1|F2|F33|F3|File|File}
1098 #v:vFNAME:{f1}
1099
1100 # check if program name is unique:
1101 # create filename patterns...
1102 r@:{(^#vFNAME#\)|(\|#vFNAME#\)|(\|#vFNAME#$)}|v:
    vFNAME_REGEX
1103 # use them to test for program existence
1104 y:vProgramsList | f*!:vFNAME_REGEX:lPROGNAME_UNIQUE:
    lPROG_EXISTS
1105 l:lPROGNAME_UNIQUE
1106 # So, program isn't registered yet, meaning its ok
1107 # to proceed to register it...
1108 # add name to programs list
1109 g*:{|}:vProgramsList:vFNAME | v:vProgramsList | v@:
    dvProgramsList
1110 r@:{User PROGRAM [#vFNAME#] is now REGISTERED.
1111 CAN BE INVOKED as SYSTEM COMMAND.}|q!:
1112
1113 l:lPROG_EXISTS
1114 r@:{User PROGRAM [#vFNAME#] Already EXISTS!
1115 CANNOT re-REGISTER PROGRAM.}|q!:
1116
1117 l:lNFINE
1118 i!:{Invalid [blank] File Name!
1119 CANNOT REGISTER PROGRAM.}
1120 "
1121
1122
1123 #programs
1124 V:fShowProgramsList:"
1125 y@:dvProgramsList|h!:{\|}|d:{\|}
1126 "
1127
1128 #programs PATTERN
1129 V:fSearchProgramsList:"
1130 v:vREGEX_PATTERN
1131 y@:dvProgramsList|h!:{\|}|d:{\|}|k*:vREGEX_PATTERN
1132 "
1133
1134 #run FILENAME
1135 V:fRunFile:"
1136 v:vFILENAME|t!.:|v:vFNAME

```

```

1137
1138 #imports...
1139 y@:dfGenDBFilename|v:fGenDBFilename
1140 y@:dfIsFileNameValid|v:fIsFileNameValid
1141
1142 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
1143 r@:{Invalid File Name [#vFNAME#]!
1144 FILE NOT EXECUTED.}|q!:
1145 l:1GOOD_FILENAME
1146
1147 # then, try to run the file...
1148 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
1149
1150 # for now, run it as a TEA program..
1151 r@:{y@:#vdFNAME#}|E:|E:
1152 "
1153
1154 #(isrunnable|canrun) FILENAME
1155 V:fProgramExists:"
1156 v:vFILENAME|t!..|v:vFNAME
1157
1158 y@:dfGenDBFilename|v:fGenDBFilename
1159 y@:dfIsFileNameValid|v:fIsFileNameValid
1160
1161 y:vFNAME | f:^$:1NFINE
1162 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
1163 r@:{Invalid File Name [#vFNAME#]!
1164 PROGRAM MAY NOT EXIST.}|q!:
1165 l:1GOOD_FILENAME
1166
1167 # fetch current programs name list..
1168 y@:dvProgramsList | v:vProgramsList
1169
1170 r@:{(^#vFNAME#\)|(\|#vFNAME#\)|(\|#vFNAME#\$)}|v:
    vFNAME_REGEX
1171 y:vProgramsList | f*:vFNAME_REGEX:1PROG_EXISTS
1172
1173 i!:{PROGRAM DOES NOT EXIST}|q!:
1174
1175 l:1PROG_EXISTS
1176 i!:{PROGRAM EXISTS}|q!:
1177
1178 l:1NFINE
1179 i!:{Invalid [blank] File Name!
1180 PROGRAM DOES NOT EXIST.}
1181 "
1182
1183 Y:fProgramExists | V@:dfProgramExists
1184
1185

```

```

1186 #unregister FILENAME
1187 V:fUnRegisterProgram:"
1188 # we assume given program file already exists!
1189 # first, grab incoming function argument...
1190 v:vFILENAME|t!.:|v:vFNAME
1191
1192 #then import functions..
1193 y@:dfGenDBFilename|v:fGenDBFilename
1194 y@:dfIsFileNameValid|v:fIsFileNameValid
1195
1196 y:vFNAME | f:^$:lNFINE
1197 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1198 r@:{Invalid File Name [#vFNAME#]!
1199 CANNOT UNREGISTER PROGRAM.}|q!:
1200 l:lGOOD_FILENAME
1201
1202 # fetch current program file name list..
1203 y@:dvProgramsList | v:vProgramsList
1204
1205 # create delete filename patterns + apply them..
1206 r@:{(^#vFNAME#\)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
1207 r@:{(\|#vFNAME#\)|}|v:vFNAME_REGEX_MID
1208 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
1209 # use it to del
1210 y:vProgramsList | d*:vFNAME_REGEX_ENDS | v:vProgramsList
1211 y:vProgramsList| d*:vFNAME_REGEX_ALL | v:vProgramsList
1212 v:vDIV:{|}
1213 r*:vProgramsList:vFNAME_REGEX_MID:vDIV | v:vProgramsList
1214
1215 # update programs list
1216 y:vProgramsList| v@:dvProgramsList
1217
1218 i!:{UNREGISTERED PROGRAM FILE: }|x*!:vFNAME
1219 q!:
1220
1221 l:lNFINE
1222 i!:{Invalid [blank] File Name!
1223 PROGRAM FILE NOT UNREGISTERED.}
1224 "
1225
1226 Y:fUnRegisterProgram | V@:dfUnRegisterProgram
1227
1228 #delete-program FILENAME
1229 V:fDeleteProgramFile:"
1230 # first, grab incoming function argument...
1231 v:vFILENAME|t!.:|v:vFNAME
1232
1233 #then import functions..
1234 y@:dfGenDBFilename|v:fGenDBFilename
1235 y@:dfIsFileNameValid|v:fIsFileNameValid

```

```

1236 y@:dfUnRegisterProgram |v:fUnRegisterProgram
1237
1238 y:vFNAME | f:^$:lNFINE # filename shouldn't be blank
1239 # file name should conform to accepted patterns
1240 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1241 r@:{Invalid File Name [#vFNAME#]!
1242 PROGRAM FILE NOT DELETED.}|q!:
1243 l:lGOOD_FILENAME
1244
1245 # fetch current file name list..
1246 y@:dvFileList | v:vFileList
1247
1248
1249 # create delete filename patterns + apply them..
1250 r@:{(^#vFNAME#\)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
1251 r@:{(\|#vFNAME#\)|)}|v:vFNAME_REGEX_MID
1252 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
1253 # use it to del
1254 y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
1255 y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
1256 v:vDIV:{|}
1257 r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList
1258
1259 # update file list
1260 y:vFileList | v@:dvFileList
1261 #also delete file record from db filesystem
1262 y:vFNAME|e*:fGenDBFilename|v:vdFNAME
1263 r@:{c!@:#vdFNAME#}|E:
1264
1265 # then unregister program file
1266 y:vFNAME | e*:fUnRegisterProgram | v:vUnRegStatus
1267
1268 r@:{DELETED PROGRAM FILE: #vFNAME#
1269 With Status: #vUnRegStatus#} | q!:
1270
1271
1272 l:lNFINE
1273 i!:{Invalid [blank] File Name!
1274 PROGRAM FILE NOT DELETED.}
1275 "
1276
1277 Y:fDeleteProgramFile|V@:dfDeleteProgramFile
1278
1279 L:lSYS_PROG_END
1280 #:{=====}
1281
1282
1283 *:{==|SYSTEM SHELL |==}
1284 L:lSYS_SHELL_START
1285 E*:fLogin

```



```

1286
1287 #---[ Main LOOP ]
1288 L:lSTART_Main
1289
1290 # present default prompt..
1291 E*:fMakePS
1292 V@:dvLastPS
1293 E*:fPROMPT
1294
1295 E*:fLogAI
1296
1297 #clean+process input..
1298 V:vIN_raw
1299 t!.:| v:vIN_clean
1300 Z:|v:vIN_clean_lc
1301
1302 #what did we get?
1303 y:vIN_clean_lc
1304
1305 #preliminary processing...
1306
1307 # a subsystem..
1308 ###((FILE System))
1309
1310 #> create FILENAME
1311 F!:^create\s+.*$:lN_CRT_FILE
1312 Y:vIN_raw|t!.:
1313 V@:dvLastCommand
1314 d:^\S+\s+|V:vFileName
1315 E*:fCreateFile | E*:fLog0
1316 J:lFIN_CMD_PROCESSING
1317 L:lN_CRT_FILE
1318
1319 #> ls | list | dir | files
1320 F!:{^(ls|list|dir|files)$}:lN_LIST_FILES
1321 V@:dvLastCommand
1322 E*:fShowFilesList | E*:fLog0
1323 J:lFIN_CMD_PROCESSING
1324 L:lN_LIST_FILES
1325
1326 #> (ls | list | dir | files | find) PATTERN
1327 F!:{^(ls|list|dir|files|find)\s+(\S+\s*)+$}:lN_SEARCH_FILES
1328 Y:vIN_raw|t!.:
1329 V@:dvLastCommand
1330 d:^\S+\s+|V:vFilePattern
1331 E*:fSearchFilesList | E*:fLog0
1332 J:lFIN_CMD_PROCESSING
1333 L:lN_SEARCH_FILES
1334
1335 #> (del | delete | rm) PATTERN

```

```

1336 F!:{^(del|delete|rm)\s+(\S+\s*)+$}:1N_DEL_FILES
1337 Y:vIN_raw|t!.:
1338 V@:dvLastCommand
1339 d:^\S+\s+|V:vFilePattern
1340 E*:fDeleteFilesList | E*:fLog0
1341 J:1FIN_CMD_PROCESSING
1342 L:1N_DEL_FILES
1343
1344 #> init FILENAME
1345 F!:^(init\s+.*$):1N_INIT_FILE
1346 Y:vIN_raw|t!.:
1347 V@:dvLastCommand
1348 d:^\S+\s+|V:vFileName
1349 E*:fInitFile | E*:fLog0
1350 J:1FIN_CMD_PROCESSING
1351 L:1N_INIT_FILE
1352
1353 #> write FILENAME
1354 F!:^(write\s+.*$):1N_WRITE_FILE
1355 Y:vIN_raw|t!.:
1356 V@:dvLastCommand
1357 d:^\S+\s+|V:vFileName
1358 E*:fWriteFile | E*:fLog0
1359 J:1FIN_CMD_PROCESSING
1360 L:1N_WRITE_FILE
1361
1362 #> (show|cat|read) FILENAME
1363 F!:{^(show|cat|read)\s+.*$}:1N_READ_FILE
1364 Y:vIN_raw|t!.:
1365 V@:dvLastCommand
1366 d:^\S+\s+|V:vFileName
1367 E*:fReadFile | E*:fLog0
1368 J:1FIN_CMD_PROCESSING
1369 L:1N_READ_FILE
1370
1371
1372 #> exists FILENAME
1373 F!:{^(exists)\s+.*$}:1N_EXISTS_FILE
1374 Y:vIN_raw|t!.:
1375 V@:dvLastCommand
1376 d:^\S+\s+|V:vFileName
1377 E*:fFileExists | E*:fLog0
1378 J:1FIN_CMD_PROCESSING
1379 L:1N_EXISTS_FILE
1380
1381
1382 #> (rename|rn|mv|move) FILENAME NEWNAME
1383 F!:{^(rename|rn|mv|move)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:1N_RENAME_FILE
1384 Y:vIN_raw|t!.:

```

```

1385 V@:dvLastCommand
1386 d:~\S+\s+|V:vFileNames
1387 Y:vFileNames | D:\s+.*$ | V:vFileName
1388 Y:vFileNames | D!:\s+.*$ | V:vNewName
1389 #check..
1390 #R@:{Renaming #vFileName# to #vNewName#} | q!:
1391 # ensure source file exists...
1392 Y:vFileName | E*:fFileExists
1393 F!:{^FILE EXISTS$}:lCANNOT_RENAME
1394
1395 # also check if it is a registered program...
1396 Y:vFileName | E*:fProgramExists
1397 F:{^PROGRAM EXISTS$}:lPROCESS_RENAME_PROGRAM
1398
1399 # renaming normal file...
1400 # read src into data buffer
1401 Y:vFileName | E*:fReadFile | v@:dvDATABUFFER
1402 # write data buffer into new file
1403 Y:vNewName | E*:fSaveDataToFile | v:vWRITELOG
1404 # delete old file
1405 Y:vFileName | E*:fDeleteFile
1406 R@:{File #vFileName# RENAMED #vNewName#
1407 With status: #vWRITELOG#}
1408 E*:fLog0
1409 J:lFIN_CMD_PROCESSING
1410
1411 L:lPROCESS_RENAME_PROGRAM
1412 # renaming program file...
1413 # read src into data buffer
1414 Y:vFileName | E*:fReadFile | v@:dvDATABUFFER
1415 # write data buffer into new file
1416 Y:vNewName | E*:fSaveDataToFile | v:vWRITELOG
1417 # register new program
1418 Y:vNewName | E*:fRegisterProgramFile | v:vREGLOG
1419 # delete old program file
1420 Y:vFileName | E*:fDeleteProgramFile | v:vDELLOG
1421 R@:{Program File #vFileName# RENAMED #vNewName#
1422 With [WRITE] status: #vWRITELOG#
1423 And [REGISTER] status: #vREGLOG#
1424 And [DEL] status: #vDELLOG#}
1425 E*:fLog0
1426 J:lFIN_CMD_PROCESSING
1427
1428 L:lCANNOT_RENAME
1429 R@:{File #vFileName# does NOT EXIST!
1430 RENAME Operation ABANDONED}
1431 E*:fLog0
1432 J:lFIN_CMD_PROCESSING
1433 L:lN_RENAME_FILE
1434

```

```

1435 #> (copy|clone) FILE1 FILE2
1436 F!:{^(copy|clone)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_
      ]+(\.[a-z0-9]+)?)$}:1N_CLONE_FILE
1437 Y:vIN_raw|t!.:
1438 V@:dvLastCommand
1439 d:^(S+\s+|V:vFileNames
1440 Y:vFileNames | D:\s+.*$ | V:vFileName
1441 Y:vFileNames | D!:\s+.*$ | V:vFileName2
1442 #check..
1443 #R@:{Cloning #vFileName# to #vFileName2#} | q!:
1444 # ensure source file exists...
1445 Y:vFileName | E*:fFileExists
1446 F!:{^FILE EXISTS$}:1CANNOT_CLONE
1447 # read src into data buffer
1448 Y:vFileName | E*:fReadFile | v@:dvDATABUFFER
1449 # write data buffer into new file
1450 Y:vFileName2 | E*:fSaveDataToFile | v:vWRITELOG
1451 R@:{File #vFileName# CLONED TO #vFileName2#
1452 With status: #vWRITELOG#}
1453 E*:fLog0
1454 J:1FIN_CMD_PROCESSING
1455
1456 1:1CANNOT_CLONE
1457 R@:{File #vFileName# does NOT EXIST!
1458 CLONE Operation ABANDONED}
1459 E*:fLog0
1460 J:1FIN_CMD_PROCESSING
1461 L:1N_CLONE_FILE
1462
1463
1464 #> export FILENAME
1465 F!:^(export\s+.*$}:1N_EXPORT_FILE
1466 Y:vIN_raw|t!.:
1467 V@:dvLastCommand
1468 #exit via file export..
1469 d:^(S+\s+|V:vFileName
1470 E*:fReadFile | v:vLOG # read file
1471 E*:fResetLog # reset log
1472 # Gracefully quit...
1473 E*:fOSHandle | X:{Thanks for Using }
1474 V:vMSG
1475 G*!:vDELIM2:vFileName:vLOG:vMSG
1476 J:1EXIT_Main
1477 L:1N_EXPORT_FILE
1478
1479 ###((END FILE System))
1480
1481 # a subsystem..
1482 ###((INTERFACE System))
1483

```

```

1484 #> uimini
1485 F!:^uimini$:1N_UIMINI
1486 V@:dvLastCommand
1487 V@:dvUIMODE_MINI:{1}
1488 I!:{TOS Mini-Interface Mode Activated} | E*:fLog0
1489 J:1FIN_CMD_PROCESSING
1490 L:1N_UIMINI
1491
1492 #> uilog
1493 F!:^uilog$:1N_UILOG
1494 V@:dvLastCommand
1495 V@:dvUIMODE_MINI:{0}
1496 I!:{TOS Log-Interface Mode Activated} | E*:fLog0
1497 J:1FIN_CMD_PROCESSING
1498 L:1N_UILOG
1499
1500 ###((END INTERFACE System))
1501
1502
1503 # a subsystem..
1504 ###((HELP System))
1505
1506 #> about
1507 F!:^about$:1N_ABOUT
1508 V@:dvLastCommand
1509 E*:fGetAboutTOS | E*:fLog0
1510 J:1FIN_CMD_PROCESSING
1511 L:1N_ABOUT
1512
1513 #> help | man | whatis | info
1514 F!:{^(help|man|whatis|info)$}:1N_HELP
1515 V@:dvLastCommand
1516 E*:fGetHelpMenu | E*:fLog0
1517 J:1FIN_CMD_PROCESSING
1518 L:1N_HELP
1519
1520 #> (help|man|whatis|info|about) NAME
1521 F!:{^(help|man|whatis|info|about)\s+(\S+\s*)+}$}:
1522     1N_HELP_TOPIC
1523 Y:vIN_raw|t!.:
1524 V@:dvLastCommand
1525 #extract+process topic..
1526 d:{^\S+\s+} | E*:fGetHelpOn | E*:fLog0
1527 J:1FIN_CMD_PROCESSING
1528 L:1N_HELP_TOPIC
1529
1530 #> help-file[s]?
1531 F!:^help-file[s]?$:1N_HELP_FILE
1532 |V@:dvLastCommand
1533 E*:fGetFileCMDMenu | E*:fLog0

```

```
1533 J:1FIN_CMD_PROCESSING
1534 L:1N_HELP_FILE
1535
1536 #> help-prog[s|rams]?
1537 F!:^help-prog([s]|rams)?$:1N_HELP_PROG
1538 V@:dvLastCommand
1539 E*:fGetProgrammingMenu | E*:fLog0
1540 J:1FIN_CMD_PROCESSING
1541 L:1N_HELP_PROG
1542
1543 #> help-util[s]?
1544 F!:^help-util[s]?$:1N_HELP_UT
1545 V@:dvLastCommand
1546 E*:fGetUtilityMenu | E*:fLog0
1547 J:1FIN_CMD_PROCESSING
1548 L:1N_HELP_UT
1549
1550 #> help-ui[s]?
1551 F!:^help-ui[s]?$:1N_HELP_UI
1552 V@:dvLastCommand
1553 E*:fGetUIMenu | E*:fLog0
1554 J:1FIN_CMD_PROCESSING
1555 L:1N_HELP_UI
1556
1557 #> help-help[s]?
1558 F!:^help-help[s]?$:1N_HELP_HELP
1559 V@:dvLastCommand
1560 E*:fGetHelpMenu
1561 X:{
1562 To use or access help on any topic in the system, use any of
    the commands presented in the following Menu.
1563 }
1564 X!:{
1565 Each of those commands leads you to further information
    about help on other commands. For example
1566
1567 > help help-util
1568
1569 Will explain what the help-util command is all about.}
1570
1571 E*:fLog0
1572 J:1FIN_CMD_PROCESSING
1573 L:1N_HELP_HELP
1574
1575 ###((END HELP System))
1576
1577 # a subsystem..
1578 ###((UTILITY System))
1579
1580 #> echo MESSAGE
```

```
1581 F!:^echo.*$:1N_ECHO
1582 Y:vIN_raw|t!.:
1583 d:^\S+\s*|V:vMESSAGE
1584 X:{echo }
1585 |V@:dvLastCommand
1586 Y:vMESSAGE
1587 E*:fLog0
1588 J:1FIN_CMD_PROCESSING
1589 L:1N_ECHO
1590
1591 #> (fetch|web) URL
1592 F!:^(fetch|web)\s+\S*$:1N_FETCH
1593 Y:vIN_raw|t!.:
1594 V@:dvLastCommand
1595 #extract url..
1596 d:^\S+\s+}
1597 E*:fFixURL
1598 W: #do the web..
1599 E*:fLog0
1600 J:1FIN_CMD_PROCESSING
1601 L:1N_FETCH
1602
1603 #> tea CODE
1604 F!:^tea\s+[\S]+.*$:1N_TEA
1605 Y:vIN_raw|t!.:
1606 #extract code..
1607 d:^\S+\s+|V:vTEACode
1608 X:{tea }
1609 |V@:dvLastCommand
1610 Y:vTEACode
1611 E: #do the tea..
1612 E*:fLog0
1613 J:1FIN_CMD_PROCESSING
1614 L:1N_TEA
1615
1616 #> math EXPRESSION
1617 F!:^math\s+[\S]+.*$:1N_MATH
1618 Y:vIN_raw|t!.:
1619 #extract expression..
1620 d:^\S+\s+|V:vMathExp
1621 X:{math }
1622 |V@:dvLastCommand
1623 Y:vMathExp
1624 R.: #do the math..
1625 E*:fLog0
1626 J:1FIN_CMD_PROCESSING
1627 L:1N_MATH
1628
1629 #> username NAME
1630 F!:^username\s+[\S]+.*$:1N_SETUSER
```

```
1631 V@:dvLastCommand
1632 #extract expression..
1633 Y:vIN_raw|t!.:
1634 d:^(S+\s+
1635 V@:dvUSERNAME #update db
1636 E*:fLog0
1637 J:1FIN_CMD_PROCESSING
1638 L:1N_SETUSER
1639
1640 #> (whoami|name)
1641 F!:{^(whoami|name)}$:1N_WHOAMI
1642 V@:dvLastCommand
1643 Y@:dvUSERNAME
1644 E*:fLog0
1645 J:1FIN_CMD_PROCESSING
1646 L:1N_WHOAMI
1647
1648
1649 #> time
1650 F!:^(time$):1N_TIME
1651 v@:dvLastCommand
1652 #show cur.time..
1653 Z.:TIME | E*:fLog0
1654 #present+log I/O
1655 J:1FIN_CMD_PROCESSING
1656 L:1N_TIME
1657
1658 #> date
1659 F!:^(date$):1N_DATE
1660 v@:dvLastCommand
1661 Z.:DATE | E*:fLog0
1662 J:1FIN_CMD_PROCESSING
1663 L:1N_DATE
1664
1665 #> timestamp
1666 F!:^(timestamp$):1N_TSTAMP
1667 v@:dvLastCommand
1668 Z.:TIMESTAMP | E*:fLog0
1669 J:1FIN_CMD_PROCESSING
1670 L:1N_TSTAMP
1671
1672 #> clear | reset
1673 F!:{^(clear|reset)}$:1N_CLEAR
1674 v@:dvLastCommand
1675 E*:fResetLog
1676 E*:fLog0
1677 J:1FIN_CMD_PROCESSING
1678 L:1N_CLEAR
1679
1680 #> print
```



```
1681 F!:^print$:LN_PRINT
1682 V@:dvLastCommand
1683 E*:fGetLatestOutputBuffer
1684 E*:fLog0
1685 J:1FIN_CMD_PROCESSING
1686 L:LN_PRINT
1687
1688 #> logout
1689 F!:^logout$:LN_LOGOUT
1690 C!:@:dvUSERNAME # delete username
1691 C*:vUSERNAME # also clear vault
1692 E*:fResetLog # reset log
1693 # ensure to clear log backup too!
1694 C@:dvLastOutput_backup
1695 # Gracefully quit...
1696 E*:fOSHandle | X:{You have SIGNED OUT!
1697 Thanks for Using }
1698 J:1EXIT_Main
1699 L:LN_LOGOUT
1700
1701 #> exit
1702 F!:^exit$:LN_EXIT
1703 #don't log input cmd.
1704 E*:fResetLog # reset log
1705 #prepare for immediate exit..
1706 # Gracefully quit...
1707 E*:fOSHandle | X:{Thanks for Using }
1708 J:1EXIT_Main
1709 L:LN_EXIT
1710
1711 #> export
1712 F!:^export$:LN_EXPORT
1713 #don't log input cmd.
1714 #exit via export..
1715 Y@:dvLOG|V:vLOG
1716 E*:fResetLog # reset log
1717 # Gracefully quit...
1718 E*:fOSHandle | X:{Thanks for Using }
1719 V:vMSG
1720 G*!:vDELIM2:vLOG:vMSG
1721 J:1EXIT_Main
1722 L:LN_EXPORT
1723
1724 ###((END UTILITY System))
1725
1726
1727 # a subsystem..
1728 ###((PROGRAMMING System))
1729
1730 #> (runnable|register|add-program) FILENAME
```

```

1731 F!:{^(runnable|register|add-program)\s+([a-z0-9_]+(\.[a-z0
      -9]+)?)$}:1N_RUNNABLE_FILE
1732 Y:vIN_raw|t!.:
1733 V@:dvLastCommand
1734 d:^\S+\s+|V:vFileName
1735 #check..
1736 #R@:{Registering User-Program in #vFileName#} | q!:
1737 # ensure source file exists...
1738 Y:vFileName | E*:fFileExists
1739 F!:{^FILE EXISTS$}:1CANNOT_REGISTER_PROG
1740 # register file as user-program..
1741 Y:vFileName | E*:fRegisterProgramFile
1742 E*:fLog0
1743 J:1FIN_CMD_PROCESSING
1744
1745 L:1CANNOT_REGISTER_PROG
1746 R@:{File #vFileName# does NOT EXIST!
1747 User-Program Registration Operation ABANDONED}
1748 E*:fLog0
1749 J:1FIN_CMD_PROCESSING
1750 L:1N_RUNNABLE_FILE
1751
1752
1753 #> programs
1754 F!:{^programs$}:1N_LIST_PROGRAMS
1755 V@:dvLastCommand
1756 E*:fShowProgramsList | E*:fLog0
1757 J:1FIN_CMD_PROCESSING
1758 L:1N_LIST_PROGRAMS
1759
1760 #>programs PATTERN
1761 F!:{^programs\s+(\S+\s*)+$}:1N_SEARCH_PROGRAMS
1762 Y:vIN_raw|t!.:
1763 V@:dvLastCommand
1764 d:^\S+\s+|V:vFilePattern
1765 E*:fSearchProgramsList | E*:fLog0
1766 J:1FIN_CMD_PROCESSING
1767 L:1N_SEARCH_PROGRAMS
1768
1769 #> (run|exec|execute) FILENAME
1770 F!:{^(run|exec|execute)\s+.*$}:1N_RUN_FILE
1771 Y:vIN_raw|t!.:
1772 V@:dvLastCommand
1773 d:^\S+\s+|V:vFileName
1774 E*:fRunFile | E*:fLog0
1775 J:1FIN_CMD_PROCESSING
1776 L:1N_RUN_FILE
1777
1778 #> (isrunnable|canrun) FILENAME
1779 F!:{^(isrunnable|canrun)\s+.*$}:1N_CANRUN_FILE

```

```

1780 Y:vIN_raw|t!.:
1781 V@:dvLastCommand
1782 d:^\S+\s+|V:vFileName
1783 E*:fProgramExists | E*:fLog0
1784 J:lFIN_CMD_PROCESSING
1785 L:lN_CANRUN_FILE
1786
1787
1788 #> (del-prog|delete-program|unregister) FILENAME
1789 F!:{^(del-prog|delete-program|unregister)\s+([a-z0-9_]+(\.[a-
-z0-9_]+)?)$}:lN_UNREGISTER_FILE
1790 Y:vIN_raw|t!.:
1791 V@:dvLastCommand
1792 d:^\S+\s+|V:vFileName
1793 #check..
1794 #R@:{Unregistering User-Program in #vFileName#} | q!:
1795 # ensure source file exists...
1796 Y:vFileName | E*:fFileExists
1797 F!:{^FILE EXISTS$}:lCANNOT_REGISTER_PROG
1798 # unregister user-program..
1799 Y:vFileName | E*:fUnRegisterProgram
1800 E*:fLog0
1801 J:lFIN_CMD_PROCESSING
1802
1803 L:lCANNOT_REGISTER_PROG
1804 R@:{File #vFileName# does NOT EXIST!
1805 User-Program Unregistration Operation ABANDONED}
1806 E*:fLog0
1807 J:lFIN_CMD_PROCESSING
1808 L:lN_UNREGISTER_FILE
1809
1810
1811 #> (update|update-program) FILE1 FILE2
1812 F!:{^(update|update-program)\s+([a-z0-9_]+(\.[a-z0-9_]+)?)\s
+([a-z0-9_]+(\.[a-z0-9_]+)?)$}:lN_UPDATE_PROGRAM
1813 Y:vIN_raw|t!.:
1814 V@:dvLastCommand
1815 d:^\S+\s+|V:vFileNames
1816 Y:vFileNames | D:\s+.*$ | V:vFileName
1817 Y:vFileNames | D!:\s+.*$ | V:vUpdateFileName
1818 #check..
1819 #R@:{Updating #vFileName# using #vUpdateFileName#} | q!:
1820
1821 # ensure source file exists...
1822 Y:vUpdateFileName | E*:fFileExists
1823 F!:{^FILE EXISTS$}:lCANNOT_READ_SRC
1824
1825 # ensure target file exists...
1826 Y:vFileName | E*:fFileExists
1827 F!:{^FILE EXISTS$}:lCANNOT_WRITE_TARGET

```

```

1828
1829 # also ensure it is a registered program...
1830 Y:vFileName | E*:fProgramExists
1831 F:{^PROGRAM EXISTS$}:lPROCESS_UPDATE_PROGRAM
1832
1833 R@:{Target File #vFileName# IS NOT a PROGRAM!
1834 Program UPDATE Operation ABANDONED}
1835 E*:fLog0
1836 J:lFIN_CMD_PROCESSING
1837
1838 L:lPROCESS_UPDATE_PROGRAM
1839 # updating program file...
1840 # read src into data buffer
1841 Y:vUpdateFileName | E*:fReadFile | v@:dvDATABUFFER
1842 # write data buffer into existing program file
1843 Y:vFileName | E*:fSaveDataToFile | v:vWRITELOG
1844
1845 R@:{Program File #vFileName# UPDATED USING #vUpdateFileName#
1846 With [WRITE] status: #vWRITELOG#}
1847 E*:fLog0
1848 J:lFIN_CMD_PROCESSING
1849
1850 L:lCANNOT_WRITE_TARGET
1851 R@:{Target File #vFileName# does NOT EXIST!
1852 Program UPDATE Operation ABANDONED}
1853 E*:fLog0
1854 J:lFIN_CMD_PROCESSING
1855
1856 L:lCANNOT_READ_SRC
1857 R@:{Source File #vUpdateFileName# does NOT EXIST!
1858 Program UPDATE Operation ABANDONED}
1859 E*:fLog0
1860 J:lFIN_CMD_PROCESSING
1861 L:lN_UPDATE_PROGRAM
1862
1863 #> PROGRAM | FILENAME
1864 F!:{^([a-z0-9_]+(\.[a-z0-9]+)?)\s*$}:lN_RUN_FILE_PROG
1865 Y:vIN_raw|t!.:
1866 V@:dvLastCommand | V:vFileName
1867
1868 # ensure it is a registered program, otherwise cnf..
1869 Y:vFileName | E*:fProgramExists
1870 F:{^PROGRAM EXISTS$}:lPROCESS_RUN_PROGRAM:lCMD_CATCHALL
1871
1872 L:lPROCESS_RUN_PROGRAM
1873 Y:vFileName | E*:fRunFile | E*:fLog0
1874 J:lFIN_CMD_PROCESSING
1875 L:lN_RUN_FILE_PROG
1876
1877 ###((END PROGRAMMING System))

```

```

1878
1879
1880 ###((CATCH-ALL))
1881 L:lCMD_CATCHALL
1882 #if we came this far
1883 #then it's CNF!
1884 F:~$:lFIN_CMD_PROCESSING
1885 Y:vIN_raw | V@:dvLastCommand # log unknown command
1886 I!:{--COMMAND NOT FOUND | INVALID COMMAND--}
1887 E*:fLogO
1888 J:lFIN_CMD_PROCESSING
1889 ###((END CATCH-ALL))
1890
1891 #we only naturally get here for properly
1892 #processed commands..
1893 L:lFIN_CMD_PROCESSING
1894
1895 #for non-exit commands, we want to ensure
1896 #we log user response from latest command..
1897 E*:fLogAI
1898
1899 J:lSTART_Main
1900 ###[End Main LOOP]
1901
1902 #outside shell.. quitting
1903 L:lEXIT_Main
1904
1905 #delete session log from db
1906 C!@:dvLOG
1907
1908 Q!: #quit+return AI
1909
1910 L:lSYS_SHELL_END
1911 #: {=====}

```

Listing 1: TEA Program: TOS v1.1.0 Complete Source-Code

6 About TEA OS v1.1.0 and TEA v1.5.4

Anyone interested in trying out not just the most stable TEA language release, but also its hottest application/use-case to date needs get their hands on the TEA OS without doubt. A preview of what has been accomplished so far can readily be accessed via the TEA WEB IDE: <https://tea.nuchwezi.com>, however, the specifics concerning that particular milestone are:

The latest official reference implementation of the TEA runtime and documentation has reached v1.5.4. That version of TEA has been robustly tested and proven, being the one currently powering the final, minimal, feature-complete,

self-documenting TEA Operating System (TEA OS) also currently being included as part of the official TTTT package; also at v1.5.4 right now. Thus, not only can one use TEA for the ‘usual’ tasks (refer to the TEA TAZ [1]), but with TEA OS, one has a complete TEA-powered file-system, a suite of basic utilities including ability to do math and test simple and complex minified TEA programs from the TOS shell terminal, and the power to write, store, and later invoke stored TEA programs as system commands directly from TEA OS.

Also, among great observations while working with TEA OS is that all work done and stored can not only be readily exported outside of the OS (per session), but also that; files create and stored inside of TEA OS can later be updated, renamed, deleted or exported outside of the TEA OS as is. Concerning this, it should be worth noting that, unlike most other operating systems (big and small), that often require some kind of virtual machine in order to be run from inside a host operating system, that the TEA OS is actually creatively designed (thanks to TEA), so that, anywhere TEA programs can run, the TEA OS can likewise be run — without need for any modern or traditional virtualization technologies in-between the TEA OS and say Linux or Windows! However, it should also be worth noting that, despite this, files created or read from inside the TEA OS aren’t immediately or readily visible to the host operating system; that is, when inside TEA OS, you can not run a command like “ls” and expect to see a listing of files hosted on the underlying operating system. Similarly, you can not be inside Windows or Linux, and expect to readily see, modify or read files created inside of the TEA OS! This should then indicate that TEA and TEA OS somewhat allow for a means of virtualization that somewhat goes beyond or is acutely different from what normal operating systems implementation might be accustomed to².

Like all TEA programs, the complete TEA OS can fit inside a string, and one can carry it as a lightweight, easier-to-use alternative to Linux, Unix or Windows/MS-DOS in just any modern browser or via the TEA mobile IDE without installing anything, no virtual machines, no sophisticated configurations... it just works!

TO try TEA OS without installing anything, use the hot-load-and-run URL:
<https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/276b40c96b52f846bf11214419912e3f/raw/&i=Please+Click+Run+In+Case+It+Does+Not+Auto+Run&run>

The latest documentation on both TEA and TEA OS can be found in [1], and as for what it feels like working inside of the TEA OS right now, the following illustrative session dump might be worth reviewing or referring to..

²Thus the joke some TEAists are currently championing, that the “TEA OS is a #beyond-linux OS”.


```
=o==o==o==o==o==o==o==
-----
TOS/21:36:02/>
about

=o==o==o==o==o==o==o==
ABOUT: TEA OS v1.1.0
#---[ABOUT TEA OS]

NAME: TEA Operating System
(also "TEA OS" or just "TOS")

ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>

SINCE: 9th APRIL, 2026

INTENT: To manifest a true general purpose, lightweight,
        minimal but powerful operating system that can
        essentially just fit inside a single string; to see
        what the limits of the mathematics of TRANSFORMATICS
        might be, and what we surely could accomplish with its
        proper use via modern portable computers.

ABOUT: TEA OS is implemented using the Transforming
        Executable Alphabet (TEA) programming language. It is
        based on a specification first laid out in the TEA TAZ.
        It is a project originating from UGANDA.

PROJECT HOME: Nuchwezi Research | https://tea.nuchwezi.com
=o==o==o==o==o==o==o==
-----
TOS/21:36:15/>
help-file

=o==o==o==o==o==o==o==
TEA OS v1.1.0 FILE System Commands:
create FILENAME
init FILENAME
write FILENAME
(rename | rn | mv | move) FILENAME NEWNAME
(copy|clone) FILE1 FILE2
exists FILENAME
```



```

find PATTERN
(show | cat | read) FILENAME
export FILENAME
(del | delete | rm) FILENAME
(ls | list | dir | files) PATTERN
=o==o==o==o==o==o==o=

-----
TOS/21:36:25/>
help-help

To use or access help on any topic in the system, use any
    of the commands presented in the following Menu.

=o==o==o==o==o==o==o=
TEA OS v1.1.0 HELP System Commands:
about
(help | man | whatis | info | about) [NAME]
help-file
help-help
help-prog
help-ui
help-util
=o==o==o==o==o==o==o=

Each of those commands leads you to further information
    about help on other commands. For example

> help help-util

Will explain what the help-util command is all about.

-----
TOS/21:36:37/>
uilog
TOS Log-Interface Mode Activated
-----
TOS/21:36:56/>
help uimini

=o==o==o==o==o==o==o=
uimini <~ Configure TEA OS interface to render using mini-
    terminal mode. Setting persists across logins until
    changed. This mode is suitable for environments such as
    using TOS on some Desktop Computer Systems.

```

```

=o==o==o==o==o==o==o=

-----
TOS/21:37:05/>
uimini
TOS Mini-Interface Mode Activated
-----
TOS/21:37:11/>
help-prog

=o==o==o==o==o==o==o=
TEA OS v1.1.0 PROGRAMMING System Commands:
(runnable | register | add-program) FILENAME
(isrunnable | canrun) FILENAME
(run | exec | execute) FILENAME
(del-prog | delete-program | unregister) FILENAME
(update | update-program) FILENAME
PROGRAM
(help | man | whatis | info | about) PROGRAM
=o==o==o==o==o==o==o=

-----
TOS/21:37:18/>
whatis program

=o==o==o==o==o==o==o=
PROGRAM <~ Executes PROGRAM as a system command if it
exists --- first, if it is any of the standard commands
in TEA OS, otherwise if it is a valid filename "
PROGRAM" that is already recognized/registered as a
user-created program - this later case similar to
calling "run PROGRAM", otherwise shows error message
command-not-found.
=o==o==o==o==o==o==o=

-----
TOS/21:37:34/>
okay
--COMMAND NOT FOUND | INVALID COMMAND--
-----
TOS/21:37:42/>
help-utils

=o==o==o==o==o==o==o=

```

```
TEA OS v1.1.0 UTILITY System Commands:
clear | reset
date
echo MESSAGE
exit
export
(fetch | web) [URL] [FILENAME]
logout
math EXPRESSION
print
tea CODE
time
timestamp
username NAME
whoami | name
=o==o==o==o==o==o==o==o=

-----
TOS/21:37:48/>
date
Monday, 18 May 2026
-----
TOS/21:37:52/>
whoami
Fut. Prof. JWJ
-----
TOS/21:37:57/>
programs
nugreetings
the_greetings
about.tea
-----
TOS/21:38:01/>
cat about.tea
i!:Hello World
-----
TOS/21:38:16/>
run about.tea
Hello World
-----
TOS/21:38:23/>
tea n!:100|v:vN|r@:{Here's a random number: #vN#}
Here's a random number: 94
-----
```

```
TOS/21:39:55/>
echo n!:100|v:vN|r@:{Here's a random number: #vN#}
n!:100|v:vN|r@:{Here's a random number: #vN#}

-----
TOS/21:40:11/>
write rng.tea
DATA WRITTEN to File: rng.tea

-----
TOS/21:40:30/>
whatis rng.tea

=o==o==o==o==o==o==o=
That help topic [rng.tea] is not understood or supported.
    Try the 'help' command to see what help commands there
    are. For example 'help whatis'
=o==o==o==o==o==o==o=

-----
TOS/21:40:38/>
programs
nugreetings
the_greetings
about.tea

-----
TOS/21:40:42/>
files
.
datefile
info.text
info
nugreetings
run
the_greetings
permute
helloworld
about.tea
rng.tea

-----
TOS/21:40:48/>
add-program rng.tea
User PROGRAM [rng.tea] is now REGISTERED.
CAN BE INVOKED as SYSTEM COMMAND.

-----
TOS/21:41:01/>
```

```

whatis rng.tea

=o==o==o==o==o==o==o=
rng.tea <~ Is a valid user-registered [TEA] program. You
    might run or execute it by directly invoking "rng.tea"
    or via "run rng.tea".
=o==o==o==o==o==o==o=

-----
TOS/21:41:07/>
run rng.tea
Here's a random number: 23

-----
TOS/21:41:16/>
about move

=o==o==o==o==o==o==o=
move FILENAME NEWNAME <~ Change the name of an existing
    file from FILENAME to NEWNAME. WILL ALSO update the
    registered [user] programs list if FILENAME was a
    registered program. Valid FILENAME should consist of
    ONLY letters a-zA-Z, numbers 0-9, underscore _ and/or a
    dot .
=o==o==o==o==o==o==o=

-----
TOS/21:41:29/>
move rng.tea rng
Program File rng.tea RENAMED rng
With [WRITE] status: DATA WRITTEN to File: rng
And [REGISTER] status: User PROGRAM [rng] is now REGISTERED
.
CAN BE INVOKED as SYSTEM COMMAND.
And [DEL] status: DELETED PROGRAM FILE: rng.tea
With Status: UNREGISTERED PROGRAM FILE: rng.tea

-----
TOS/21:41:41/>
whatis rng

=o==o==o==o==o==o==o=
rng <~ Is a valid user-registered [TEA] program. You might
    run or execute it by directly invoking "rng" or via "
    run rng".
=o==o==o==o==o==o==o=

```

```
-----
TOS/21:41:47/>
rang
--COMMAND NOT FOUND | INVALID COMMAND--
-----
TOS/21:41:50/>
rng
Here's a random number: 60
-----
TOS/21:41:53/>
rng
Here's a random number: 76
-----
TOS/21:41:58/>
time
21:42:04
-----
TOS/21:42:04/>
name
Fut. Prof. JWJ
-----
TOS/21:42:07/>
export
=o==o==o==o==o==o==o==o==o=
Thanks for Using TEA OS v1.1.0
```

8 Conclusion

The Transforming Executable Alphabet has made possible the implementation of a minimal, lightweight, portable and feature-complete software operating environment known as the TEA OS or TOS. TEA OS demonstrates that a compact, high-level runtime and service stack can provide many of the conveniences of an operating system while remaining extremely portable and runnable in environments where traditional kernels cannot [readily] operate. We characterize TEA OS as a minimal OS runtime rather than a conventional kernel; extending it to full kernel status would require adding low-level primitives (boot, drivers, memory protection, scheduling) or a small privileged shim, a direction we identify as future work in case the need should arise, but otherwise which, based on the design and original intentions of TOS, we might not have to worry about.

We have covered highlights of TOS v1.1.0 in this paper, including looking at the underlying system architecture, the complete verbatim source-code of “the operating system that can fit inside of a single string”, as well as looked at a sam-

ple session dump exported from TOSH. The future is very bright! Given TOS can be run without the need for any traditional virtualization technologies such as VirtualBox, Hypervisor, ISOs, etc. It means that one can indeed leverage TEA OS to not only learn how to work inside a standard minimal (terminal-UI operating system), but also that, work typically heavy to carry around even for so-called lightweight operating systems such as Minix, Linux-on-A-Stick/USB, etc, cannot readily rival the ease with which one can create, update, run, post, pull, and export files as well as programs using the TEA operating system from the most unusual of places — inside a web browser, on a typical mobile phone/smartphone! Further work concerning informing the world (both in academia, but also the industry) about what has been accomplished with TEA, but also the TEA OS is much needed, however, and without doubt, the future, or what some used to think of as “the future”, is indeed, finally, already here!

References

- [1] Joseph Willrich Lutalo. **TEA TAZ - Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION**. *Academia.edu*, 2024.
- [2] Joseph Willrich Lutalo. **TEA: Transforming Executable Alphabet - A Minimal Language Leveraging Sequence Transformer Chaining**, May 2026. <https://doi.org/10.5281/zenodo.20094948>.
- [3] Joseph Willrich Lutalo, Odongo Steven Eyobu, and Benjamin Kanagwa. **DNAP: Dynamic Nuchwezi Architecture Platform-A New Software Extension and Construction Technology**. *authorea*, 2023.
- [4] Ellen Siever, Stephen Spainhour, Stephen Figgins, and Jessica P. Hekman. *Linux in a Nutshell: A Desktop Quick Reference*. O'Reilly Media, Sebastopol, CA, 3rd edition, 2000. Available online at <https://learning.oreilly.com/library/view/linux-in-a-nutshell/0596000251/>.
- [5] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, Madison, WI, 2018. Free PDF available at the official site: <https://pages.cs.wisc.edu/~remzi/OSTEP/>.
- [6] Richard Y. Kain. *Advanced Computer Architecture: A Systems Design Approach*. Prentice Hall, Englewood Cliffs, NJ, 1999. Digital preview available at Internet Archive: <https://archive.org/details/advancedcomputer0000kain>.
- [7] Microsoft Copilot. Research assistance and various clarifications. AI-generated insights via Copilot discussion, 2026. Personal communication, during drafting and revisions of manuscript.